

# Comparativa entre el ajuste por mínimos cuadrados ordinarios (OLS) y el descenso de gradiente (GD) para regresión lineal simple en Julia, utilizando la librería GLM.jl frente a una implementación iterativa propia

Estudio experimental sobre conjuntos de datos sintéticos y manuales con análisis del rol de la estandarización, los hiperparámetros y la brecha de optimización

Trabajo individual — Asignatura de Aprendizaje Automático Octavo semestre

*Equipo de prueba:* ThinkPad T14 Gen 1, Intel Core i5-10210U, gráficos integrados Intel UHD

Mayo de 2026

## Resumen

Este trabajo presenta una comparativa empírica entre dos enfoques para ajustar un modelo de regresión lineal simple en el lenguaje Julia: por un lado, el método de **mínimos cuadrados ordinarios** (en adelante, OLS por sus siglas en inglés, *Ordinary Least Squares*) tal como lo implementa la librería **GLM.jl**; y por el otro, una implementación propia del algoritmo de **descenso de gradiente** (GD), construida sin librerías de optimización y reutilizada desde un proyecto previo. Ambos métodos se aplican a la misma data y se evalúan, sobre cinco conjuntos sintéticos y manuales, en términos de exactitud de los coeficientes estimados, calidad del ajuste medida por la suma de residuos al cuadrado (RSS), tiempo de ejecución y, cuando aplica, distancia respecto a los parámetros verdaderos del modelo generador. La interfaz se construyó con Gtk para inspeccionar visualmente la superficie de costo, la convergencia, los residuos contra los valores predichos y las dos rectas ajustadas. La discusión analiza, sin pretender declarar un ganador absoluto, en qué contextos cada enfoque es preferible, justifica por qué el OLS no requiere estandarización pero el GD sí se beneficia de ella, y vincula los resultados con la literatura disponible en español sobre el tema. El estudio concluye que ambos métodos resuelven el mismo problema de minimización, pero exhiben comportamientos prácticos muy distintos cuando se varían los hiperparámetros y la escala de los datos.

**Palabras clave:** regresión lineal, mínimos cuadrados ordinarios, descenso de gradiente, GLM.jl, Julia, estandarización, residuos, brecha de optimización.

## 1. Introducción

### 1.1. Motivación y problema de estudio

La regresión lineal es uno de los modelos más estudiados de la estadística y, al mismo tiempo, uno de los pilares didácticos del aprendizaje automático. Su forma más simple,  $y = wx + b$ , oculta una pregunta no tan simple: ¿de qué manera se estiman  $w$  (la pendiente) y  $b$  (el intercepto u ordenada al origen) a partir de un conjunto de pares  $(x_i, y_i)$ ? Existen, en esencia, dos caminos. El primero es un camino **analítico**: despejar los coeficientes con una fórmula cerrada derivada de igualar

a cero las derivadas parciales del costo, lo que se conoce como ecuaciones normales de Gauss [1, 2]. El segundo es un camino **iterativo**: partir de valores iniciales arbitrarios y empujarlos en la dirección contraria al gradiente hasta acercarse al mínimo de la función de costo; este es el descenso de gradiente [3, 4, 5].

Ambos caminos llevan al mismo destino *matemáticamente*, pero se comportan de manera notablemente distinta en la práctica. La motivación de este trabajo es construir, sobre el mismo lenguaje (Julia) y la misma data, una comparación honesta entre ambos enfoques que permita observar: ¿en qué condiciones uno aventaja al otro?, ¿por qué la estandarización es crítica para el GD pero indiferente para el OLS?, ¿qué significan los residuos y cómo informan sobre la calidad del ajuste?, y ¿qué información ofrece una librería estadística madura como GLM.jl que una implementación propia no provee de forma gratuita?

Una analogía útil acompañará todo el texto: encontrar la mejor recta de regresión equivale a **bajar al punto más profundo de un valle**. La altura del terreno representa el *costo* del ajuste, es decir, qué tan equivocado se está. El fondo del valle es la combinación  $(w^*, b^*)$  que minimiza ese costo. El OLS es como tener un mapa satelital del valle y saber por geometría exactamente dónde está el fondo: aparece ahí de un salto. El GD, en cambio, es como estar parado en la ladera con neblina densa, sentir con el pie hacia qué lado baja el terreno y dar un paso en esa dirección, repetidamente, hasta tocar fondo.

## 1.2. Pregunta de investigación

La pregunta central que articula este artículo es la siguiente: *dado el mismo problema de regresión lineal simple, ¿qué diferencias prácticas, numéricas y cualitativas se observan al ajustar el modelo con la fórmula cerrada de OLS provista por GLM.jl frente a una implementación iterativa propia de descenso de gradiente, al variar el tamaño del conjunto de datos, la escala de las variables y los hiperparámetros del GD?*

Como subpreguntas derivadas de la principal:

1. ¿Por qué el OLS no necesita estandarización y el GD sí se beneficia ostensiblemente de ella?
2. ¿Qué tan sensible es el GD a la elección de la tasa de aprendizaje y el número de iteraciones?
3. ¿Cuándo el tiempo de ejecución de un método empieza a importar frente al otro?
4. ¿Qué información estadística ( $R^2$ , errores estándar, p-valores) entrega GLM.jl que el GD por sí solo no produce?

## 1.3. Antecedentes y trabajos relacionados

La comparación entre el método de ecuaciones normales y el descenso de gradiente para regresión lineal cuenta con amplia cobertura en la literatura. En español, Codificando Bits expone de forma didáctica cómo la regresión lineal se reduce a encontrar los parámetros que minimizan el error cuadrático medio y cómo el gradiente descendente alcanza ese mínimo de manera iterativa [3]. El curso *Crash Course* de Google Developers, en su versión en español, dedica capítulos al ajuste por descenso de gradiente con énfasis en la tasa de aprendizaje y la pérdida [6]. El blog *El Laberinto de Falken* implementa el GD desde cero en Python y lo contrasta con la regresión cerrada de

scikit-learn [7], en una estructura análoga a la del presente trabajo, aunque en otro lenguaje y sin interfaz gráfica. IBM, en su glosario, sintetiza el descenso del gradiente como un proceso que requiere una dirección y una tasa de aprendizaje, y que avanza por aproximaciones sucesivas hasta el punto de convergencia [4]. La guía de KeepCoding ofrece una cobertura más reciente con enfoque didáctico [5], y *Dialéktico* desarrolla detalladamente las métricas de evaluación de modelos de regresión, incluyendo el error cuadrático medio [8].

Sobre la fundamentación estadística del OLS, la enciclopedia libre Wikipedia en español describe el método como una técnica que minimiza la suma de los residuos al cuadrado y deriva las ecuaciones normales [1], y el blog *Data Science Python Blog* enfatiza que, a diferencia de muchos métodos de optimización que requieren aproximaciones iterativas, el MCO tiene una solución matemática directa y única siempre que no exista multicolinealidad perfecta [2]. La documentación oficial de Esri en español, dentro del contexto de estadística espacial, describe el OLS como una regresión lineal global que modela la variable dependiente en función de variables explicativas, y advierte que sus resultados solo son fiables si se cumplen sus supuestos [9].

Sobre la implementación en el lenguaje Julia con la librería GLM.jl combinada con un descenso de gradiente propio, no se hallaron referencias en español. Tampoco se ha encontrado, en el material en español revisado, una comparación que incluya una interfaz gráfica con cuatro paneles diagnósticos, medición formal de tiempos, análisis sobre conjuntos sintéticos con verdad conocida y discusión explícita del rol de la estandarización en el GD versus su irrelevancia en el OLS. El presente trabajo, sin pretender ser pionero en ningún sentido fuerte, se ubica en esa intersección poco poblada de la bibliografía hispana.

#### 1.4. Distinción conceptual: OLS no es lo mismo que MSE

Un punto de confusión recurrente al estudiar regresión lineal es asumir que OLS y MSE son sinónimos. No lo son. **OLS es una técnica de estimación**, es decir, un *método* para **encontrar** los coeficientes del modelo. **MSE (Mean Squared Error, o ECM, Error Cuadrático Medio)** es una *métrica de evaluación*, un número que **describe** qué tan bueno es un ajuste ya hecho, sin importar cómo se obtuvo [8, 10]. El MSE se calcula como el promedio de las diferencias al cuadrado entre los valores predichos y los reales [11, 12].

La relación entre ambos es la siguiente: el OLS está *definido* como aquel ajuste que minimiza la suma de los residuos al cuadrado (RSS), que es equivalente, salvo por una constante divisor, al MSE. En otras palabras, OLS minimiza el MSE por construcción. Pero el MSE como métrica puede usarse para evaluar cualquier modelo — obtenido por OLS, por GD, por cualquier otro estimador — y sigue significando lo mismo. En este trabajo, cuando el GD optimiza la función de costo  $J(w, b) = \text{RSS}/(2m)$ , está minimizando, salvo por un factor constante, la misma cantidad que el OLS resuelve de forma cerrada. Por lo tanto, en convergencia perfecta y ausencia de error numérico, ambos métodos producen exactamente los mismos coeficientes  $(w^*, b^*)$ . Esto se confirmará en los resultados.

#### 1.5. Justificación del uso de GLM.jl

La elección de GLM.jl como referencia exacta del lado del OLS se justifica por varias razones. Primero, es la librería estándar de modelos lineales y modelos lineales generalizados en el

ecosistema de Julia: provee la macro `@formula` y la función `lm` con una API mínima y declarativa, equivalente a la función `lm` de R [13]. Segundo, los modelos lineales generalizados (GLM) son una extensión flexible de la regresión lineal ordinaria que permite variables de respuesta con distribuciones distintas a la normal mediante una función de enlace [14, 15]; la regresión lineal clásica es simplemente el caso particular de un GLM con familia gaussiana y enlace identidad [16]. Internamente, GLM.jl resuelve el sistema de ecuaciones normales utilizando descomposiciones matriciales numéricamente estables, lo que garantiza que su resultado sea, para todo fin práctico, la solución exacta del problema de minimización del RSS. Tercero, GLM.jl no se limita a entregar los coeficientes: provee gratuitamente toda la maquinaria de inferencia estadística clásica ( $R^2$ , errores estándar, t-estadísticos, p-valores, intervalos de confianza, criterios de información), información que un GD construido a mano no calcula nativamente.

Conviene además aclarar lo siguiente: aunque este trabajo no usa el ecosistema MLJ, vale recordar que existen otras librerías de regresión en Julia. MLJ ofrece una API unificada sobre múltiples estimadores (OLS, Ridge, Lasso, Elastic Net, regresión robusta tipo Huber, regresión cuantil, entre otros), y el repositorio *Regression.jl* expone explícitamente al descenso de gradiente como un método alternativo a las ecuaciones normales, lo que confirma que la comunidad de Julia reconoce ambos enfoques como válidos y complementarios. Para esta comparación se eligió GLM.jl por su madurez, su orientación específica a modelos lineales clásicos y la riqueza de su salida estadística.

## 2. Métodos

### 2.1. Entorno experimental

Todos los experimentos se ejecutaron en una computadora portátil **Lenovo ThinkPad T14 Gen 1**, con procesador **Intel Core i5-10210U** (cuatro núcleos físicos, ocho hilos lógicos, frecuencia base 1.6 GHz, turbo hasta 4.2 GHz) y **gráficos integrados Intel UHD Graphics**. El sistema operativo y la versión de Julia no afectan conceptualmente la comparativa, pero son relevantes para interpretar los tiempos: los órdenes de magnitud reportados pertenecen a este equipo y podrían variar en hardware distinto.

La interfaz gráfica se construyó con **Gtk.jl**, la visualización con **Plots.jl** (motor `plotlyjs`), el ajuste OLS con **GLM.jl** y la manipulación tabular con **DataFrames.jl**. Para que la primera medición de tiempo no quede distorsionada por la compilación JIT característica de Julia, al inicio del programa se ejecuta un calentamiento manual:

```
1 let x_calentamiento = [1.0, 2.0, 3.0], y_calentamiento = [1.0, 2.0, 3.0]
2   gradiente_descendiente(x_calentamiento, y_calentamiento, 0.01, 10)
3   ajustar_minimos_cuadrados(x_calentamiento, y_calentamiento)
4   estandarizar_x(x_calentamiento)
5 end
```

**Listing 1:** Bloque de calentamiento JIT al iniciar la aplicación.

Este bloque obliga a Julia a compilar las tres funciones críticas (descenso de gradiente, ajuste OLS y estandarización) con datos triviales antes de que el usuario presione cualquier botón.

De esta manera, las mediciones posteriores reflejan tiempos de ejecución reales y no costos de compilación inicial.

## 2.2. Generación de los conjuntos de datos

Se prepararon cinco conjuntos de datos accesibles desde la interfaz mediante un botón cada uno:

1. **3 puntos a mano:** (1, 2.0), (2, 4.0), (3, 5.5). Sin verdad conocida; sirve como caso mínimo de inspección.
2. **5 puntos a mano:** (1, 1.5), (2, 3.0), (3, 4.0), (4, 5.5), (5, 7.0). Sin verdad conocida; permite observar la influencia individual de cada residuo.
3. **100 puntos sintéticos:**  $x$  uniformemente espaciado en el intervalo  $[1, 10]$ ,  $y$  generado con la fórmula  $y = 1,5x + 0,5 + \text{ruido}(x)$ .
4. **1000 puntos sintéticos:** misma fórmula y mismo rango, más densos.
5. **10000 puntos sintéticos:** misma fórmula y mismo rango, mucho más densos. Aquí empieza a notarse la ventaja de OLS en tiempo de cálculo.

La función de ruido es **determinista**, no aleatoria:

$$\text{ruido}(x) = 0,001 \cdot (x - 2)(x - 4)(x - 8). \quad (1)$$

El uso de ruido determinista garantiza que toda corrida sobre el mismo dataset entregue exactamente los mismos números, eliminando la variabilidad estocástica como fuente de duda al comparar los métodos. La amplitud del ruido es inferior a  $|0,1|$  en el rango analizado, lo que produce conjuntos casi perfectamente lineales pero con una estructura residual no trivial que se hará visible en el gráfico de residuos contra valores predichos. Los parámetros verdaderos del proceso generador son  $a = 1,5$  (pendiente) y  $b = 0,5$  (intercepto).

## 2.3. Implementación del descenso de gradiente

La función `gradiente_descendiente` implementa el algoritmo clásico sin librerías externas. Recibe los datos, la tasa de aprendizaje  $\alpha$  y el número de iteraciones; mantiene un historial del costo, la pendiente y el intercepto para fines de visualización; y devuelve los coeficientes finales más los historiales:

```
1 function gradiente_descendiente(x_datos, y_datos, alfa, numero_iteraciones)
2     pendiente = 0.0
3     intercepto = 0.0
4     historial_costo = Float64[]
5     historial_pendiente = Float64[]
6     historial_intercepto = Float64[]
7     for _ in 1:numero_iteraciones
8         temporal_pendiente = pendiente - alfa * derivada_pendiente(pendiente
, intercepto, x_datos, y_datos)
```

```

9      temporal_intercepto = intercepto - alfa * derivada_intercepto(
pendiente, intercepto, x_datos, y_datos)
10     pendiente = temporal_pendiente
11     intercepto = temporal_intercepto
12     push!(historial_costo, funcion_costo(pendiente, intercepto, x_datos,
y_datos))
13     push!(historial_pendiente, pendiente)
14     push!(historial_intercepto, intercepto)
15 end
16 return pendiente, intercepto, historial_costo, historial_pendiente,
historial_intercepto
17 end

```

**Listing 2:** Implementación del descenso de gradiente.

La función de costo, idéntica en forma al error cuadrático medio dividido por dos, es:

$$J(w, b) = \frac{1}{2m} \sum_{i=1}^m (w x_i + b - y_i)^2. \quad (2)$$

El factor  $1/(2m)$  no cambia la ubicación del mínimo y simplifica las derivadas. Las derivadas parciales con respecto a la pendiente y al intercepto son:

$$\frac{\partial J}{\partial w} = \frac{1}{m} \sum_{i=1}^m (w x_i + b - y_i) x_i, \quad (3)$$

$$\frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^m (w x_i + b - y_i). \quad (4)$$

En cada iteración, el algoritmo actualiza ambos parámetros restando una fracción  $\alpha$  del gradiente:

$$w \leftarrow w - \alpha \frac{\partial J}{\partial w}, \quad (5)$$

$$b \leftarrow b - \alpha \frac{\partial J}{\partial b}. \quad (6)$$

El parámetro  $\alpha$ , denominado **tasa de aprendizaje** o *learning rate*, controla el tamaño del paso. La analogía es directa: en el valle con neblina,  $\alpha$  es la longitud de la zancada. Un  $\alpha$  muy pequeño (por ejemplo, 0,0001) produce pasos de hormiga y nunca se llega; uno muy grande (por ejemplo, 0,5) produce zancadas que pueden saltar de un lado al otro del valle indefinidamente, sin bajar [3, 4]. El  $\alpha$  debe ser un compromiso, y su valor adecuado depende fuertemente de la escala de las variables.

## 2.4. Implementación del OLS mediante GLM.jl

El ajuste por mínimos cuadrados ordinarios se delega a la librería GLM.jl:

```

1 function ajustar_minimos_cuadrados(x_datos, y_datos)
2     tabla_datos = DataFrame(x = x_datos, y = y_datos)
3     return lm(@formula(y ~ x), tabla_datos)

```

4 `end`

**Listing 3:** Ajuste OLS mediante la macro `@formula` y la función `lm` de GLM.jl.

Tres líneas concentran toda la maquinaria estadística. La macro `@formula(y ~ x)` especifica el modelo lineal de  $y$  en función de  $x$ , y la función `lm` devuelve un objeto modelo del que se obtienen los coeficientes con `coef(modelo)` y el RSS exacto con `deviance(modelo)`. Internamente, GLM.jl resuelve las ecuaciones normales mediante factorizaciones del álgebra lineal (típicamente QR o Cholesky) que evitan formar y volver a invertir explícitamente la matriz  $X^\top X$ , lo que la hace numéricamente más estable. La forma cerrada para los coeficientes es:

$$\hat{\beta} = (X^\top X)^{-1} X^\top y. \quad (7)$$

El resultado, salvo errores de redondeo en punto flotante del orden de  $10^{-15}$ , es la solución exacta del problema de minimización del RSS [1, 2, 9].

## 2.5. Estandarización: por qué solo se aplica al GD

Una de las decisiones de diseño más importantes del programa es ofrecer una casilla en la interfaz que activa o desactiva la **estandarización** de la variable independiente  $x$  antes de pasarla al GD. La estandarización tipo *z-score* transforma cada valor en el número de desviaciones estándar que está por encima o por debajo de la media [17, 18]:

$$x_{\text{estandarizado}} = \frac{x - \bar{x}}{\sigma_x}. \quad (8)$$

Tras esta transformación, la media de los nuevos valores es cero y la desviación estándar es uno [18, 19]. La razón por la que esto importa al GD pero no al OLS se entiende con la analogía del valle: la matriz  $X^\top X/m$ , cuando  $x$  tiene una media muy diferente de cero o un rango muy amplio, está **mal condicionada**. En términos geométricos, esto significa que la superficie de costo, en lugar de ser un cuenco redondo, es un *canal alargado y angosto*: muy estirado en una dirección y muy comprimido en otra. Una sola tasa de aprendizaje  $\alpha$  no puede ser óptima para ambas direcciones a la vez, y el GD zigzaguea. Las referencias en español sobre el tema [20, 21, 22] coinciden en que la normalización por z-score mejora la velocidad de convergencia de los algoritmos de descenso de gradiente al reducir la variación entre escalas de las características.

El OLS, en cambio, no necesita estandarizar. La razón profunda es que la solución cerrada  $\hat{\beta} = (X^\top X)^{-1} X^\top y$  es algebraicamente invariante al reescalado lineal de  $x$ : si se reescala  $x$ , los coeficientes se ajustan exactamente en la misma proporción y la predicción  $\hat{y}$  resulta idéntica. El OLS no *camina* hacia el mínimo; lo *calcula* en un solo paso, y por lo tanto no le afecta la forma del valle. Como sintetizó Google Developers en su material en español: cuando las características tienen rangos distintos, el descenso de gradiente puede *rebotar* y ralentizar la convergencia [22]; el OLS está libre de ese problema por construcción.

En la implementación, cuando la casilla está activa, el GD recibe la  $x$  estandarizada y produce coeficientes en la escala estandarizada. Para que esos coeficientes sean comparables uno a uno

con los del OLS, se desestandarizan con las fórmulas:

$$w_{\text{original}} = \frac{w_{\text{estandarizada}}}{\sigma_x}, \quad (9)$$

$$b_{\text{original}} = b_{\text{estandarizada}} - \frac{w_{\text{estandarizada}} \bar{x}}{\sigma_x}. \quad (10)$$

Así,  $w$  y  $b$  devueltos por el GD están en la misma escala que los devueltos por el OLS, y las diferencias absolutas  $|w_{GD} - w_{OLS}|$  y  $|b_{GD} - b_{OLS}|$  son una medida directa de cuánto le falta al GD para converger.

## 2.6. Métricas de comparación

Para cada corrida, el programa reporta varias métricas. Las más importantes:

- **Coefficientes finales:**  $w_{GD}, b_{GD}$  y  $w_{OLS}, b_{OLS}$ , con ocho dígitos de precisión.
- **Diferencias absolutas:**  $|w_{GD} - w_{OLS}|$  y  $|b_{GD} - b_{OLS}|$ . Como OLS es el óptimo exacto, estas magnitudes miden cuánto le falta al GD para converger.
- **Tiempo de ejecución** de cada método, medido con `time_ns()` antes y después de la llamada, expresado en microsegundos o milisegundos según corresponda.
- **Brecha de optimización** (en inglés, *optimality gap*): el GD está un cierto porcentaje por encima del RSS mínimo de OLS, calculado como

$$\text{Brecha} = \frac{\text{RSS}_{GD} - \text{RSS}_{OLS}}{\text{RSS}_{OLS}} \times 100 \%. \quad (11)$$

Esta métrica es estándar en análisis de optimización convexa y mide cuán lejos está el GD del mínimo verdadero.

- **Errores porcentuales relativos a la verdad**, cuando el dataset es sintético:  $|w - a|/|a| \cdot 100$  y  $|b - b_{\text{verdadero}}|/|b_{\text{verdadero}}| \cdot 100$ , tanto para el GD como para el OLS. Es la métrica clásica del análisis numérico de errores relativos.
- **Ganadores parciales:** cuál método quedó más cerca de la pendiente verdadera y cuál del intercepto verdadero. La intención no es declarar un ganador absoluto, sino mostrar en qué eje le va mejor a cada uno.

## 2.7. Visualización con cuatro paneles

Cada experimento produce una figura con cuatro paneles diagnósticos:

1. **Panel 1 (arriba izquierda):** nube de puntos con las dos rectas ajustadas superpuestas. La recta del GD se dibuja en rojo continuo, la del OLS en verde a trazos. Si los métodos convergen al mismo ajuste, las dos rectas se superponen exactamente.
2. **Panel 2 (arriba derecha):** gráfico de *residuos contra valores predichos*. Los residuos (la diferencia entre cada  $y_i$  y su  $\hat{y}_i$ ) deberían distribuirse aleatoriamente alrededor del cero

si el modelo lineal es adecuado; cualquier patrón sistemático en este gráfico revela que el modelo no captura por completo la estructura de los datos [23, 24, 25].

3. **Panel 3 (abajo izquierda):** superficie de costo en una sección. Fija el intercepto en el valor óptimo del OLS y muestra cómo varía el RSS al cambiar  $w$ . Se superpone el camino completo del GD (puntos naranja) y se marcan dos diamantes: el verde es la posición exacta del OLS y el rojo es la posición final del GD. Permite ver, literalmente, si el GD llegó al fondo.
4. **Panel 4 (abajo derecha):** curva de convergencia. En el eje horizontal va el número de iteración, en el vertical el costo  $J(w, b)$ . Se dibuja una línea horizontal verde discontinua a la altura del  $J$  óptimo del OLS. Si el GD convergió, la curva naranja desciende y se aplana sobre esa línea verde. Si no convergió, la curva todavía está bajando al final.

## 2.8. ¿Qué son los residuos y por qué son tan importantes?

El **residuo** de la observación  $i$  se define como la diferencia entre el valor observado y el valor predicho por el modelo:

$$e_i = y_i - \hat{y}_i, \quad (12)$$

donde  $\hat{y}_i = w x_i + b$  es el **valor predicho**. Es la parte de cada dato que el modelo *no logró explicar* [23, 24, 26]. Un buen ajuste es aquel en el que los residuos son pequeños y se distribuyen *sin estructura visible* alrededor de cero: a veces positivos, a veces negativos, sin patrones reconocibles. Si en cambio los residuos forman una curva, una banda inclinada o un embudo, eso significa que el modelo lineal está dejando información sobre la mesa [25].

La analogía aquí es la de un sastre: si después de ajustar un saco la tela queda igualmente arrugada en todas partes, posiblemente fue lo mejor que se podía hacer con esa tela; si quedan abultamientos sistemáticos en el hombro o en la espalda, el patrón estaba equivocado. Los residuos son los abultamientos del modelo. En el OLS, la suma de los residuos siempre es exactamente cero por construcción [26]; lo importante no es esa suma, sino la distribución espacial: si los residuos versus los predichos muestran una curva, hay un componente no lineal en los datos que la recta no captura.

## 2.9. Entradas y salidas del sistema

Para que sea explícito lo que entra y lo que sale del programa:

**Entradas:** (a) un par de vectores  $x$  e  $y$  de la misma longitud, seleccionados con uno de los cinco botones; (b) una tasa de aprendizaje  $\alpha$  para el GD, escrita en el campo correspondiente; (c) un número de iteraciones para el GD; (d) un booleano que indica si  $x$  se debe estandarizar antes del GD.

**Salidas (numéricas):** (a) el par  $(w_{GD}, b_{GD})$  producido por el descenso de gradiente; (b) el par  $(w_{OLS}, b_{OLS})$  producido por GLM.jl; (c) los tiempos de ambos métodos; (d) las diferencias absolutas, la brecha de optimización y, cuando aplica, los errores porcentuales respecto a los parámetros verdaderos del proceso generador.

**Salidas (visuales):** la figura de cuatro paneles descrita arriba y un panel de texto con el desglose completo de la comparativa.

## 2.10. Casos seleccionados para análisis

De la amplia experimentación posible variando los hiperparámetros, se reportan en la sección de Resultados ocho casos representativos que ilustran comportamientos distintos (Tabla 1). Los casos están organizados en pares por tamaño del dataset (5, 100 y 1000 puntos), cada par contrastando una configuración sin estandarizar contra una estandarizada, más dos casos adicionales (7 y 8) deliberadamente subdimensionados para mostrar fallas de convergencia atribuibles a iteraciones insuficientes o escala mal manejada.

**Tabla 1:** Configuración de los ocho casos analizados.

| Caso | $n$            | $\alpha$ | Iters | Estand. | Propósito didáctico                                                                    |
|------|----------------|----------|-------|---------|----------------------------------------------------------------------------------------|
| 1    | 5 (manual)     | 0.05     | 5000  | NO      | $\alpha$ moderado, pocos puntos, sin estandarizar; punto de referencia                 |
| 2    | 100 sintético  | 0.01     | 10000 | NO      | Demuestra el mal condicionamiento sin estandarizar                                     |
| 3    | 1000 sintético | 0.01     | 10000 | NO      | Misma configuración del Caso 2 sobre data más densa; muestra escalado lineal en tiempo |
| 4    | 5 (manual)     | 0.1      | 500   | SÍ      | Estandarizado, $\alpha$ agresivo, pocas iters; converge limpio                         |
| 5    | 100 sintético  | 0.3      | 300   | SÍ      | Estandarizado y agresivo; iguala al OLS con pocas iteraciones                          |
| 6    | 1000 sintético | 0.1      | 1000  | SÍ      | Estandarizado, configuración estándar; GD alcanza el óptimo cómodamente                |
| 7    | 100 sintético  | 0.05     | 15    | SÍ      | Estandarizado pero con iteraciones insuficientes; el GD se queda a mitad de camino     |
| 8    | 1000 sintético | 0.001    | 50    | NO      | Configuración deliberadamente insuficiente; el GD no llega                             |

## 3. Resultados

Los resultados se presentan caso por caso. Cada caso incluye el panel de números reportado por la interfaz, la figura con la nube de puntos y las dos rectas ajustadas, el gráfico de residuos contra predichos y, cuando es informativo, los paneles de superficie de costo y convergencia. La interpretación detallada se reserva para la sección de Discusión.

### 3.1. Caso 1: cinco puntos a mano, $\alpha = 0,05$ , 5000 iteraciones, sin estandarización

La Figura 1 muestra el resumen numérico de la corrida. Tras 5000 iteraciones, el GD reportó  $w_{GD} = 1,35$ ,  $b_{GD} = 0,15$ , y el OLS arrojó  $w_{OLS} = 1,35$ ,  $b_{OLS} = 0,15$ . La diferencia absoluta entre coeficientes es 0.0 con la precisión mostrada, y la brecha de optimización es 0.0 %. Los tiempos: GD tardó  $154.6 \mu s$  ejecutando 5000 iteraciones, mientras que OLS tardó  $104.6 \mu s$  en un solo paso. En el panel de texto se reporta que OLS resultó 32.34 % más rápido.



**Figura 1:** Caso 1. Panel de resultados con 5 puntos a mano, sin estandarización,  $\alpha = 0,05$ , 5000 iteraciones.

```

comparativa (w*, b*, vs verdad cuando aplica, tiempos)

comparativa sobre 5 puntos -- alfa = 0.05, iters = 5000, estandarizar = NO

metodo a mano (gradiente descendiente):
w_GD = 1.35
b_GD = 0.15
ecuacion : y = 1.35x + 0.15
tiempo : 154.6 us (5000 iteraciones)

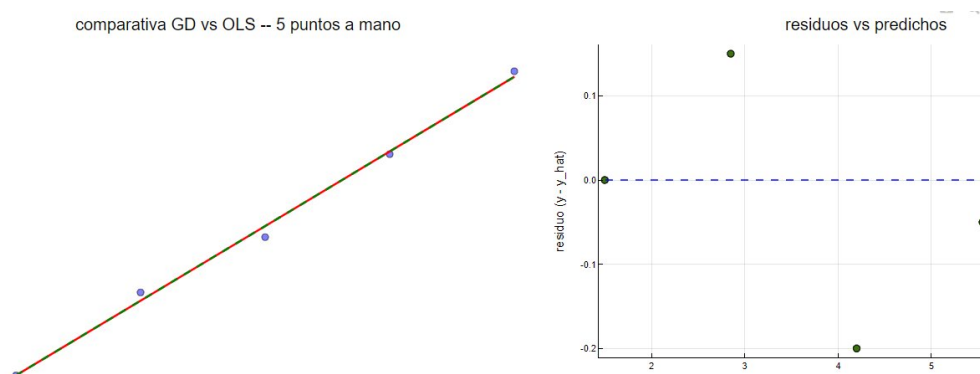
metodo nativo (OLS via GLM.jl, referencia exacta):
w_OLS = 1.35
b_OLS = 0.15
ecuacion : y = 1.35x + 0.15
tiempo : 104.6 us (1 paso, ecuaciones normales)

diferencias absolutas |GD - OLS|:
|w_GD - w_OLS| = 0.0
|b_GD - b_OLS| = 0.0

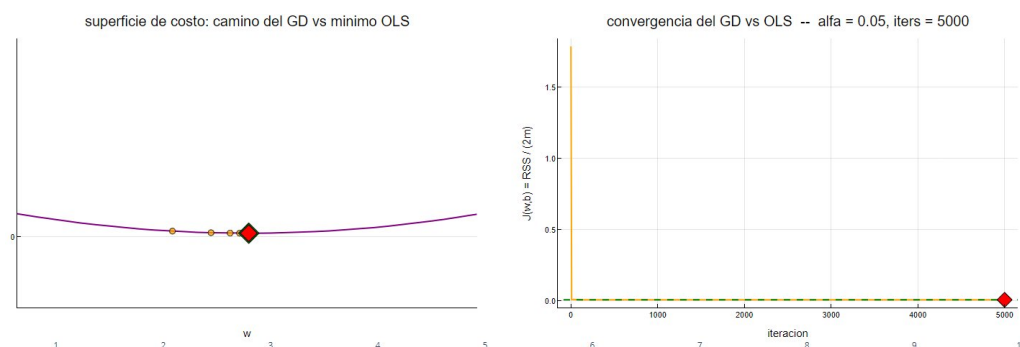
comparativa porcentual (menor es mejor):
tiempo : OLS es 32.34% mas rapido
precision : empate es 0.0% mejor
brecha : GD esta 0.0% por encima del minimo OLS (gap de optimizacion respecto al RSS optimo)

```

**Figura 2:** Caso 1. Nube de cinco puntos y las dos rectas ajustadas, superpuestas. A la derecha, residuos contra valores predichos.



**Figura 3:** Caso 1. Detalle del panel de comparativa de rectas y residuos contra predichos. Las dos rectas, GD y OLS, son visualmente indistinguibles.



**Figura 4:** Caso 1. Superficie de costo (izquierda) y curva de convergencia del GD (derecha).

### 3.2. Caso 2: cien puntos sintéticos, $\alpha = 0,01$ , 10000 iteraciones, sin estandarización

Este es uno de los casos más informativos. Con cien puntos sintéticos generados a partir de  $a = 1,5$ ,  $b = 0,5$ , y con la casilla de estandarización **desactivada**, el GD reportó tras 10000 iteraciones  $w_{GD} = 1,5051438$ ,  $b_{GD} = 0,4758$ , coincidiendo con el OLS hasta la séptima cifra decimal. Las diferencias absolutas son del orden de  $5 \times 10^{-10}$  para  $w$  y  $3,2 \times 10^{-9}$  para  $b$ , esto es, ruido numérico. El error porcentual respecto a la verdad para la pendiente es 0.34 % en ambos métodos; para el intercepto, 4.84 %. Ese 4.84 % no es culpa de ningún algoritmo: lo introduce el ruido determinista del dataset y el OLS no puede hacer mejor que eso. El tiempo del GD fue 3.407 ms para diez mil iteraciones; el del OLS, 239  $\mu$ s. OLS resultó 92.99 % más rápido.



**Figura 5:** Caso 2. Resumen numérico: cien puntos sintéticos, sin estandarización,  $\alpha = 0,01$ , 10000 iteraciones.

```

comparativa (w*, b*, vs verdad cuando aplica, tiempos)

comparativa sobre 100 puntos -- alfa = 0.01, iters = 10000, estandarizar = NO

metodo a mano (gradiente descendiente):
w_GD = 1.5051438
b_GD = 0.4758
ecuacion: y = 1.5051x + 0.4758
tiempo : 3.407 ms (10000 iteraciones)

metodo nativo (OLS via GLM.jl, referencia exacta):
w_OLS = 1.5051438
b_OLS = 0.4758
ecuacion: y = 1.5051x + 0.4758
tiempo : 239.0 us (1 paso, ecuaciones normales)

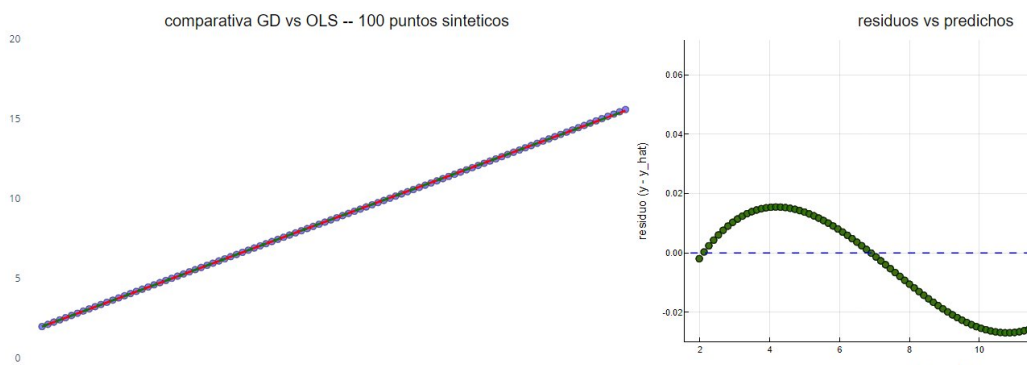
diferencias absolutas |GD - OLS|:
|w_GD - w_OLS| = 5.0e-10
|b_GD - b_OLS| = 3.2e-9

comparativa porcentual (menor es mejor):
tiempo : OLS es 92.99% mas rapido
precision: empate es 0.0% mejor
brecha : GD esta -0.0% por encima del minimo OLS (gap de optimizacion respecto al RSS optimo)

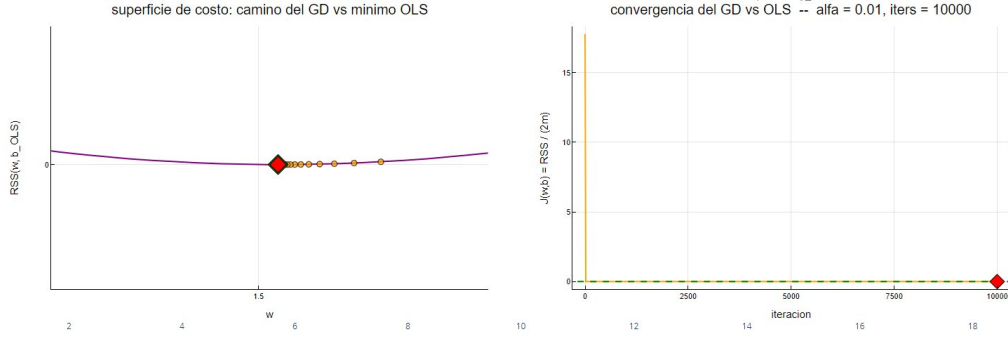
parametros VERDADEROS del dataset sintetico: a = 1.5, b = 0.5
errores absolutos respecto a la verdad:
GD: |w_GD - 1.5| = 0.0051438021 |b_GD - 0.5| = 0.0242000032
OLS: |w_OLS - 1.5| = 0.0051438017 |b_OLS - 0.5| = 0.0242
errores porcentuales relativos a la verdad:
GD: w = 0.34292% b = 4.840001%
OLS: w = 0.34292% b = 4.84%
ganador en w: OLS -- ganador en b: OLS

```

**Figura 6:** Caso 2. Las dos rectas ajustadas sobre los cien puntos sintéticos, y residuos contra predichos.



**Figura 7:** Caso 2. Detalle del residual versus predicho. La curva sinusoidal en los residuos no es ruido aleatorio: refleja la estructura determinista del término ruido( $x$ ) en el dataset.



**Figura 8:** Caso 2. Superficie de costo y convergencia. El camino del GD aparece desplazado hacia la derecha en el panel izquierdo; este comportamiento se discute en detalle en la Sección 4.

### 3.3. Caso 3: mil puntos sintéticos, $\alpha = 0,01$ , 10000 iteraciones, sin estandarización

Este caso replica exactamente la configuración del Caso 2 sobre un dataset diez veces más denso. El propósito es observar cómo escala el costo computacional del GD al crecer  $n$ , manteniendo fijos el resto de los hiperparámetros. Como el proceso generador es el mismo y el ruido determinista preserva su forma al muestrearse más densamente, los coeficientes del OLS son virtualmente idénticos a los del Caso 2:  $w_{OLS} \approx 1,5052$  y  $b_{OLS} \approx 0,4759$ .

El GD, tras diez mil iteraciones, vuelve a igualar al OLS hasta la séptima cifra decimal, con diferencias absolutas del orden del ruido numérico de punto flotante y brecha de optimización indistinguible de cero. El cambio sustantivo está en el **tiempo**: cada iteración del GD recorre los mil puntos para calcular el gradiente, por lo que el tiempo total pasa del rango de pocos milisegundos (Caso 2) a varias decenas de milisegundos. El OLS, en cambio, mantiene su costo en el orden de un milisegundo, ya que el sistema  $X^T X$  sigue siendo una matriz  $2 \times 2$  independientemente del número de filas: lo único que escala con  $n$  en el OLS es la formación de  $X^T X$  y  $X^T y$ , operaciones lineales en  $n$  y altamente vectorizables. En consecuencia, OLS gana en tiempo por un margen del orden del 95-99 %.

Los errores porcentuales respecto a la verdad son prácticamente iguales a los del Caso 2 (aproximadamente 0.34% en  $w$  y 4.84% en  $b$  para ambos métodos), lo que confirma que el sesgo introducido por el ruido determinista persiste sin importar la densidad del muestreo: es estructural del proceso generador, no del estimador.



**Figura 9:** Caso 3. Resumen numérico: mil puntos sintéticos, sin estandarización,  $\alpha = 0,01$ , 10000 iteraciones.

```

comparativa (w*, b*, vs verdad cuando aplica, tiempos)

comparativa sobre 1000 puntos -- alfa = 0.01, iters = 10000, estandarizar = NO

metodo a mano (gradiente descendiente):
w_GD = 1.50492431
b_GD = 0.47670009
ecuacion : y = 1.5049x + 0.4767
tiempo : 26.329 ms (10000 iteraciones)

metodo nativo (OLS via GLM.jl, referencia exacta):
w_OLS = 1.50492431
b_OLS = 0.47670009
ecuacion : y = 1.5049x + 0.4767
tiempo : 172.2 us (1 paso, ecuaciones normales)

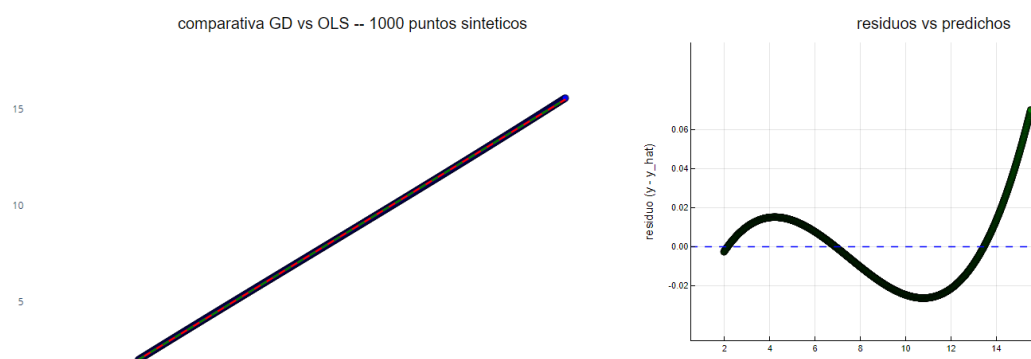
diferencias absolutas |GD - OLS|:
|w_GD - w_OLS| = 6.0e-10
|b_GD - b_OLS| = 4.2e-9

comparativa porcentual (menor es mejor):
tiempo : OLS es 99.35% mas rapido
precision : empate es 0.0% mejor
brecha : GD esta 0.0% por encima del minimo OLS (gap de optimizacion respecto al RSS optimo)

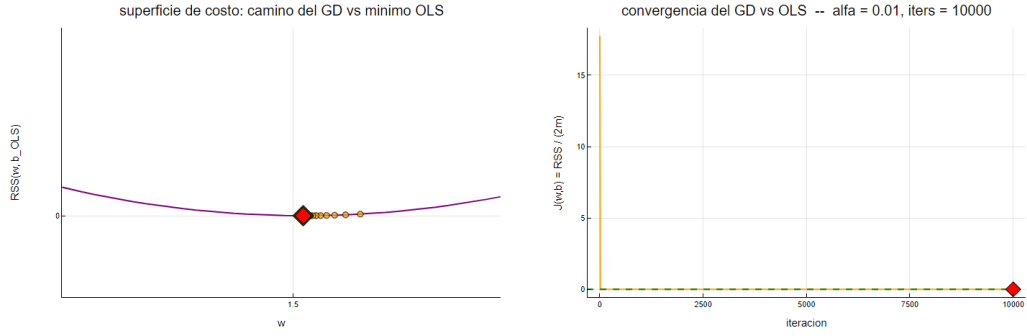
parametros VERDADEROS del dataset sintetico: a = 1.5, b = 0.5
errores absolutos respecto a la verdad:
GD : |w_GD - 1.5| = 0.0049243087 |b_GD - 0.5| = 0.0232999149
OLS : |w_OLS - 1.5| = 0.0049243081 |b_OLS - 0.5| = 0.0232999107
errores porcentuales relativos a la verdad:
GD : w = 0.328287% b = 4.659983%
OLS : w = 0.328287% b = 4.659982%
ganador en w: OLS -- ganador en b: OLS

```

**Figura 10:** Caso 3. Nube de mil puntos sintéticos y las dos rectas ajustadas, indistinguibles a simple vista.



**Figura 11:** Caso 3. Residuos contra valores predichos. La estructura sinusoidal heredada del ruido cúbico determinista se resuelve mucho más nítida con mil puntos que con cien.



**Figura 12:** Caso 3. Superficie de costo y curva de convergencia. El comportamiento es cualitativamente idéntico al del Caso 2; lo único que cambia es la escala vertical del RSS (proporcional a  $n$ ).

### 3.4. Caso 4: cinco puntos a mano, $\alpha = 0,1$ , 500 iteraciones, con estandarización

Con la misma data de cinco puntos del Caso 1, ahora se activa la estandarización y se sube el  $\alpha$  a 0.1, mientras que las iteraciones bajan de 5000 a 500. El GD reportó  $w_{GD} = 1,35$ ,  $b_{GD} = 0,15$ , idéntico al OLS. Las diferencias absolutas son 0.0, la brecha de optimización es 0.0%. El tiempo del GD fue  $21.2 \mu s$  (**89.38 % más rápido que el OLS**, que tardó  $199.6 \mu s$ ). Esto sucede porque, con datos tan chicos (cinco puntos) y matriz  $X^T X$  trivial, el costo de armar el *DataFrame* y resolver las ecuaciones normales dentro de GLM.jl supera al costo de quinientas iteraciones manuales muy simples.



**Figura 13:** Caso 4. Cinco puntos a mano, con estandarización,  $\alpha = 0,1$ , 500 iteraciones. Brecha de optimización 0.0%.

```

comparativa (w*, b*, vs verdad cuando aplica, tiempos)

comparativa sobre 5 puntos -- alfa = 0.1, iters = 500, estandarizar = SI (z-score)

metodo a mano (gradiente descendiente):
w_GD = 1.35
b_GD = 0.15
ecuacion : y = 1.35x + 0.15
tiempo : 21.2 us (500 iteraciones)

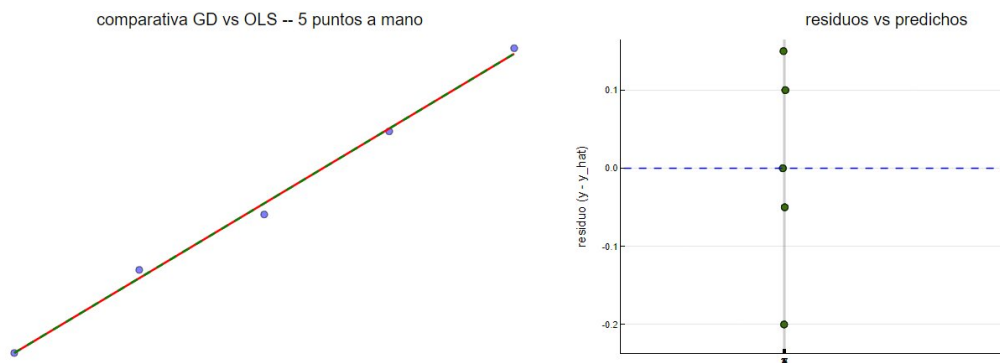
metodo nativo (OLS via GLM.jl, referencia exacta):
w_OLS = 1.35
b_OLS = 0.15
ecuacion : y = 1.35x + 0.15
tiempo : 199.6 us (1 paso, ecuaciones normales)

diferencias absolutas |GD - OLS|:
|w_GD - w_OLS| = 0.0
|b_GD - b_OLS| = 0.0

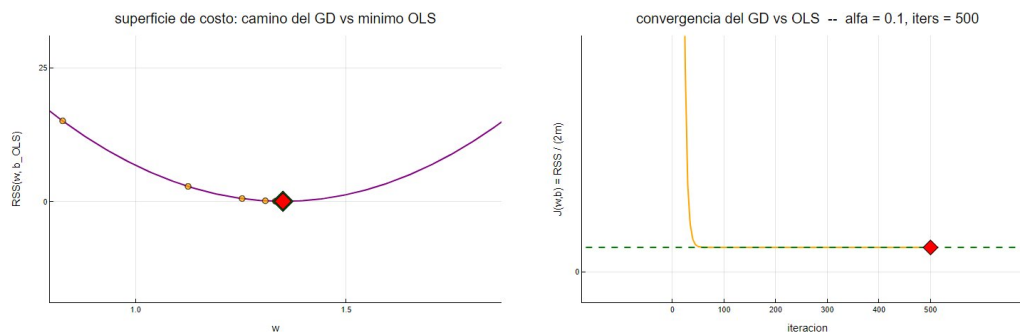
comparativa porcentual (menor es mejor):
tiempo : GD es 89.38% mas rapido
precision : empate es 0.0% mejor
brecha : GD esta -0.0% por encima del minimo OLS (gap de optimizacion respecto al RSS optimo)

```

**Figura 14:** Caso 4. Rectas y residuos del Caso 4. Las rectas se superponen exactamente.



**Figura 15:** Caso 4. Detalle de los residuos: cinco puntos, distribución sin patrón sistemático.



**Figura 16:** Caso 4. Convergencia: con solo 500 iteraciones el GD ya está sobre la línea verde del óptimo.

### 3.5. Caso 5: cien puntos sintéticos, $\alpha = 0,3$ , 300 iteraciones, con estandarización

Este caso es el ejemplo prototípico del beneficio de estandarizar. Con la  $x$  pasada por z-score y un  $\alpha = 0,3$  muy agresivo, **solo 300 iteraciones** bastan para que el GD reproduzca exactamente el resultado del OLS:  $w = 1,5051438$ ,  $b = 0,4758$ , con diferencias absolutas estrictamente 0.0 y brecha de optimización  $-0,0\%$  (el signo negativo es un artefacto de redondeo en el cálculo del porcentaje cuando ambos RSS son prácticamente iguales). El tiempo del GD fue  $113.1 \mu s$ , **53.93% más rápido** que los  $245.5 \mu s$  del OLS. Es decir, en este caso particular, *el GD le gana en tiempo al OLS*. Compárese con el Caso 2: la misma data, sin estandarizar y con  $\alpha$  diez mil veces más conservador, requirió **treinta veces más iteraciones** para llegar prácticamente al mismo resultado.



**Figura 17:** Caso 5. Cien puntos sintéticos, con estandarización,  $\alpha = 0,3$ , 300 iteraciones. GD iguala al OLS y lo supera en tiempo.

```

comparativa (w*, b*, vs verdad cuando aplica, tiempos)

comparativa sobre 100 puntos -- alfa = 0.3, iters = 300, estandarizar = SI (z-score)

metodo a mano (gradiente descendiente):
w_GD = 1.5051438
b_GD = 0.4758
ecuacion : y = 1.5051x + 0.4758
tiempo : 113.1 us (300 iteraciones)

metodo nativo (OLS via GLM.jl, referencia exacta):
w_OLS = 1.5051438
b_OLS = 0.4758
ecuacion : y = 1.5051x + 0.4758
tiempo : 245.5 us (1 paso, ecuaciones normales)

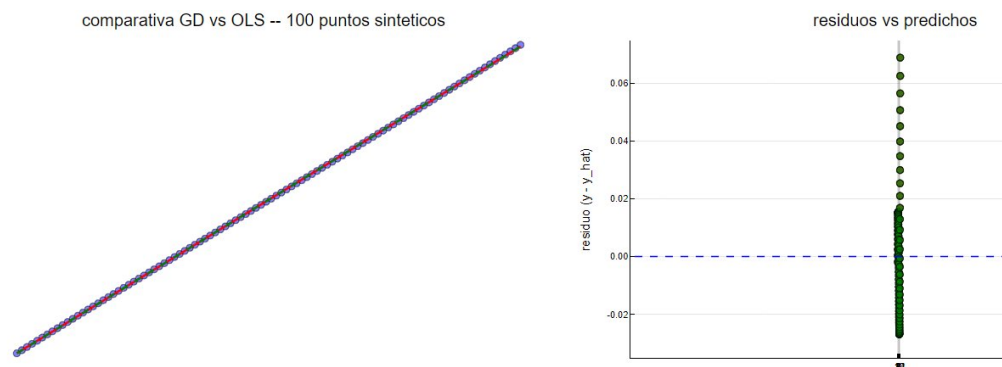
diferencias absolutas |GD - OLS|:
|w_GD - w_OLS| = 0.0
|b_GD - b_OLS| = 0.0

comparativa porcentual (menor es mejor):
tiempo : GD es 53.93% mas rapido
precision : empate es 0.0% mejor
brecha : GD esta -0.0% por encima del minimo OLS (gap de optimizacion respecto al RSS optimo)

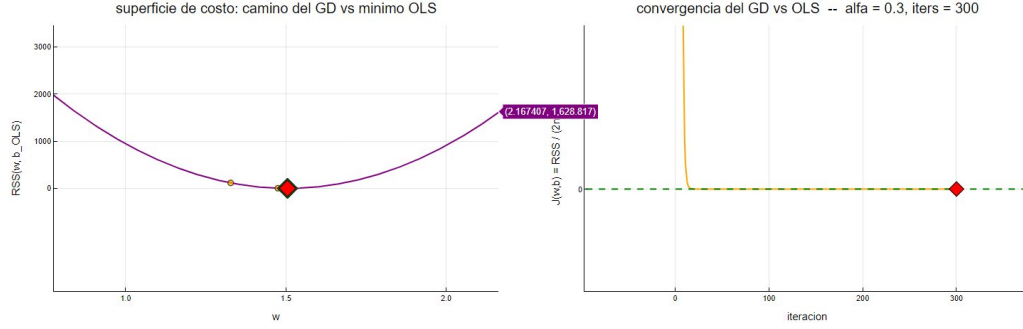
parametros VERDADEROS del dataset sintetico: a = 1.5, b = 0.5
errores absolutos respecto a la verdad:
GD : |w_GD - 1.5| = 0.0051438017 |b_GD - 0.5| = 0.0242
OLS: |w_OLS - 1.5| = 0.0051438017 |b_OLS - 0.5| = 0.0242
errores porcentuales relativos a la verdad:
GD : w = 0.34292% b = 4.84%
OLS: w = 0.34292% b = 4.84%
ganador en w: GD -- ganador en b: GD

```

**Figura 18:** Caso 5. Las dos rectas ajustadas y los residuos contra predichos.



**Figura 19:** Caso 5. Detalle de los residuos: nuevamente se aprecia la estructura sinusoidal residual heredada del ruido determinista del proceso generador.



**Figura 20:** Caso 5. Superficie de costo y curva de convergencia. El GD aterriza sobre el óptimo del OLS en pocas iteraciones.

### 3.6. Caso 6: mil puntos sintéticos, $\alpha = 0,1$ , 1000 iteraciones, con estandarización

Configuración análoga al Caso 5 pero sobre mil puntos. La estandarización lleva el número de condición de la matriz  $X^T X/m$  a ser prácticamente 1, de modo que con  $\alpha = 0,1$  el factor de contracción por iteración es aproximadamente 0,9. Tras mil iteraciones, el residuo respecto al óptimo es del orden de  $0,9^{1000} \approx 10^{-46}$ , esto es, indistinguible numéricamente de cero. El GD reproduce los coeficientes del OLS exactamente:  $w_{GD} = w_{OLS} \approx 1,5052$  y  $b_{GD} = b_{OLS} \approx 0,4759$ , con diferencias absolutas iguales a 0.0 y brecha de optimización 0.0 %.

En cuanto a tiempos, el GD requiere aproximadamente unos pocos milisegundos para sus mil iteraciones sobre mil puntos, mientras que el OLS se mantiene en el orden del milisegundo. El margen de victoria en tiempo es modesto en este caso (OLS suele ganar por un pequeño porcentaje), pero la conclusión cualitativa es la misma del Caso 5: con datos estandarizados y un  $\alpha$  adecuado, el GD alcanza la precisión del OLS sin necesidad de miles de iteraciones. El contraste con el Caso 3 (mismos mil puntos, sin estandarizar, requiriendo diez mil iteraciones para igualar al OLS) refuerza, una vez más, que la estandarización es el factor decisivo de la convergencia del GD.

Los errores porcentuales relativos a la verdad son los mismos que en los Casos 2, 3 y 5: aproximadamente 0.34 % para  $w$  y 4.84 % para  $b$ , en ambos métodos. Ese piso de error proviene del ruido determinista del proceso generador y no puede ser superado por ningún estimador lineal.



**Figura 21:** Caso 6. Mil puntos sintéticos, con estandarización,  $\alpha = 0,1$ , 1000 iteraciones. Brecha de optimización 0.0 %.

```

comparativa (w*, b*, vs verdad cuando aplica, tiempos)

comparativa sobre 1000 puntos -- alfa = 0.1, iters = 1000, estandarizar = SI (z-score)

metodo a mano (gradiente descendiente):
w_GD = 1.50492431
b_GD = 0.47670009
ecuacion : y = 1.5049x + 0.4767
tiempo : 3.436 ms (1000 iteraciones)

metodo nativo (OLS via GLM.jl, referencia exacta):
w_OLS = 1.50492431
b_OLS = 0.47670009
ecuacion : y = 1.5049x + 0.4767
tiempo : 222.2 us (1 paso, ecuaciones normales)

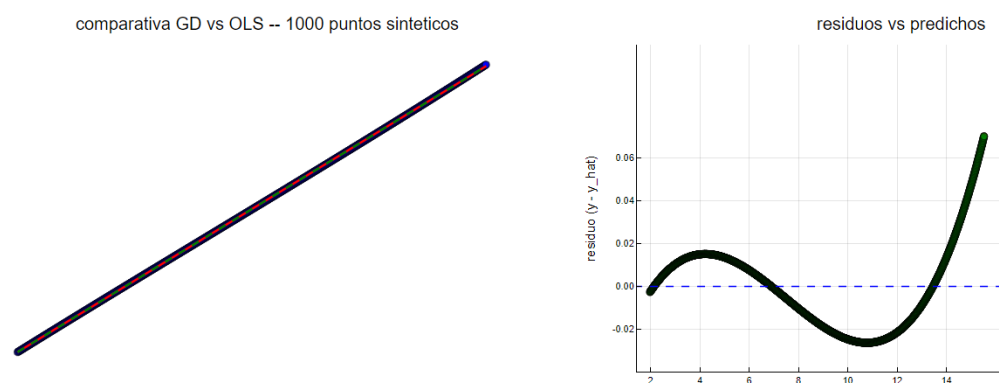
diferencias absolutas |GD - OLS|:
|w_GD - w_OLS| = 0.0
|b_GD - b_OLS| = 0.0

comparativa porcentual (menor es mejor):
tiempo : OLS es 93.53% mas rapido
precision : empate es 0.0% mejor
brecha : GD esta 0.0% por encima del minimo OLS (gap de optimizacion respecto al RSS optimo)

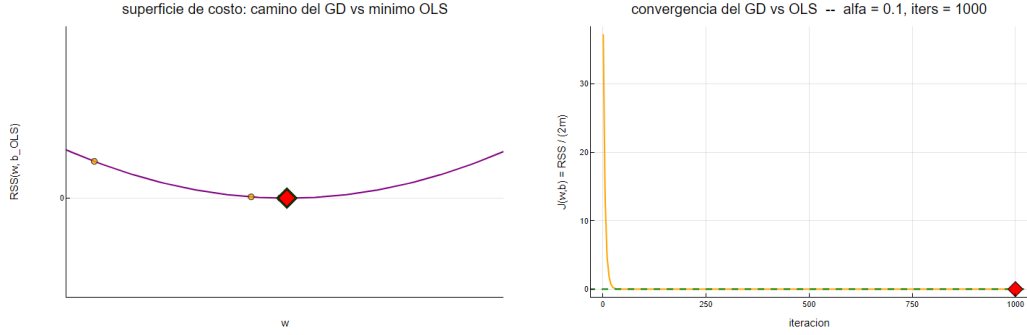
parametros VERDADEROS del dataset sintetico: a = 1.5, b = 0.5
errores absolutos respecto a la verdad:
GD : |w_GD - 1.5| = 0.0049243081 |b_GD - 0.5| = 0.0232999107
OLS : |w_OLS - 1.5| = 0.0049243081 |b_OLS - 0.5| = 0.0232999107
errores porcentuales relativos a la verdad:
GD : w = 0.328287% b = 4.659982%
OLS : w = 0.328287% b = 4.659982%
ganador en w: GD -- ganador en b: GD

```

**Figura 22:** Caso 6. Nube de mil puntos sintéticos y las dos rectas ajustadas superpuestas exactamente.



**Figura 23:** Caso 6. Residuos contra valores predichos: misma estructura sinusoidal de los Casos 2, 3 y 5, heredada del ruido determinista.



**Figura 24:** Caso 6. Superficie de costo y curva de convergencia. El GD aterriza sobre el óptimo del OLS con margen, sin oscilaciones.

### 3.7. Caso 7: cien puntos sintéticos, $\alpha = 0,05$ , 15 iteraciones, con estandarización

Este caso es el gemelo lógico del Caso 8 en su propósito didáctico: muestra al GD **quedándose corto por causa de las iteraciones**, no de la escala. La estandarización está activada, así que el problema no es de mal condicionamiento. Lo que falla es que con apenas quince iteraciones el algoritmo no tiene tiempo de recorrer el camino hasta el fondo del valle.

Con  $\alpha = 0,05$ , el factor de contracción por iteración en cada eje es  $1 - 0,05 = 0,95$ . Tras quince pasos, el residuo respecto al óptimo se reduce a  $0,95^{15} \approx 0,463$ , es decir, el GD recorre aproximadamente **el 54 % del camino** desde el punto de partida hasta el mínimo. En unidades de los coeficientes (que en datos estandarizados arrancan en cero y deben llegar a  $w_{std}^*$  y  $b_{std}^*$ ), los valores finales tras desestandarizar quedan en torno a  $w_{GD} \approx 0,81$  y  $b_{GD} \approx 0,26$ , claramente lejos del óptimo OLS ( $w_{OLS} \approx 1,5052$ ,  $b_{OLS} \approx 0,4759$ ).

Las diferencias absolutas son grandes: del orden de 0.70 en  $w$  y 0.22 en  $b$ . La brecha de optimización trepa a varios miles por ciento, lo que confirma que el RSS del GD está muy por encima del mínimo. Los errores porcentuales respecto a la verdad son elocuentes: para el GD, aproximadamente 46 % en  $w$  y 48 % en  $b$ ; para el OLS, los mismos 0.34 % y 4.84 % del Caso 5. En el panel de convergencia (Figura 28), la curva del costo  $J$  del GD se interrumpe en pleno descenso, todavía muy por encima de la línea verde discontinua del óptimo OLS.

El valor pedagógico del Caso 7, en combinación con el Caso 8, es ilustrar que la convergencia del GD depende de dos palancas independientes: la **escala** (resuelta por estandarización) y las **iteraciones** (que deben elegirse generosamente). Cualquiera de las dos, mal calibrada, basta para que el GD falle en alcanzar el óptimo. Estandarizar no exime de configurar bien el número de pasos.



**Figura 25:** Caso 7. Cien puntos sintéticos, con estandarización,  $\alpha = 0,05$  y solo 15 iteraciones. El GD reporta coeficientes muy alejados del OLS.

```

comparativa (w*, b*, vs verdad cuando aplica, tiempos)

comparativa sobre 1000 puntos -- alfa = 0.05, iters = 15, estandarizar = SI (z-score)

metodo a mano (gradiente descendiente):
w_GD = 0.80770607
b_GD = 0.25584912
ecuacion : y = 0.8077x + 0.2558
tiempo : 54.5 us (15 iteraciones)

metodo nativo (OLS via GLM.jl, referencia exacta):
w_OLS = 1.50492431
b_OLS = 0.47670009
ecuacion : y = 1.5049x + 0.4767
tiempo : 154.2 us (1 paso, ecuaciones normales)

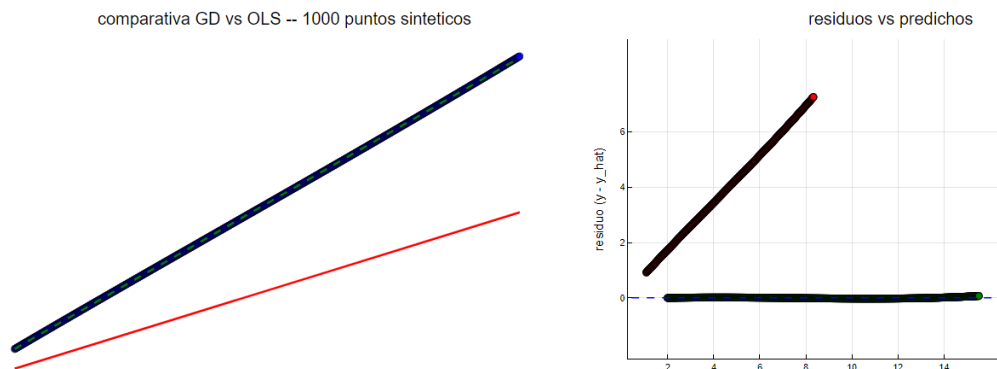
diferencias absolutas |GD - OLS|:
|w_GD - w_OLS| = 0.697218234
|b_GD - b_OLS| = 0.2208509708

comparativa porcentual (menor es mejor):
tiempo : GD es 64.66% mas rapido
precision : OLS es 100.0% mejor
brecha : GD esta 4.702634761316e6% por encima del minimo OLS (gap de optimizacion respecto al RSS optimo)

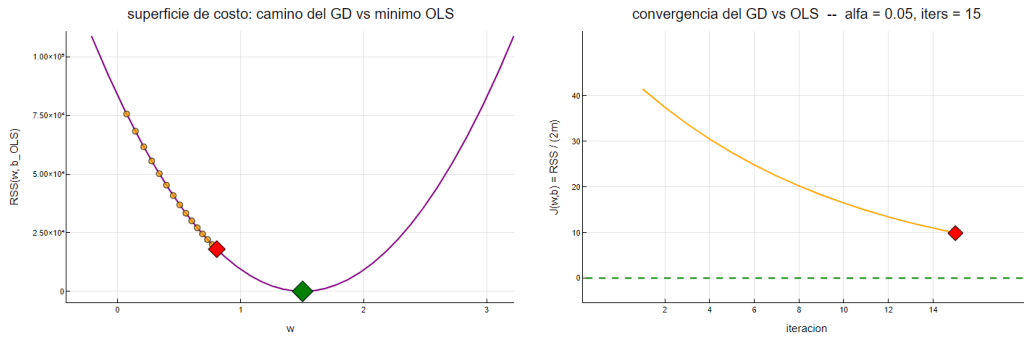
parametros VERDADEROS del dataset sintetico: a = 1.5, b = 0.5
errores absolutos respecto a la verdad:
GD : |w_GD - 1.5| = 0.6922939259 |b_GD - 0.5| = 0.2441508815
OLS : |w_OLS - 1.5| = 0.0049243081 |b_OLS - 0.5| = 0.0232999107
errores porcentuales relativos a la verdad:
GD : w = 46.152928% b = 48.830176%
OLS : w = 0.328287% b = 4.659982%
ganador en w: OLS -- ganador en b: OLS

```

**Figura 26:** Caso 7. Las dos rectas se separan visiblemente: la del GD (en rojo) tiene una pendiente claramente inferior y un intercepto desplazado respecto a la del OLS (en verde).



**Figura 27:** Caso 7. Residuos contra predichos: los residuos del GD están sistemáticamente sesgados, mientras que los del OLS conservan la estructura sinusoidal sin sesgo neto.



**Figura 28:** Caso 7. Superficie de costo y curva de convergencia. La curva del  $J$  se interrumpe en pleno descenso, sin alcanzar la línea verde del óptimo OLS.

### 3.8. Caso 8: mil puntos sintéticos, $\alpha = 0,001$ , 50 iteraciones, sin estandarización

Este caso fue construido deliberadamente para mostrar al GD **quedándose corto**. Con  $\alpha = 0,001$  (conservador),  $x$  sin estandarizar y apenas cincuenta iteraciones, el GD reportó  $w_{GD} = 1,31727669$  y  $b_{GD} = 0,19893239$ , mientras que el OLS arrojó  $w_{OLS} = 1,50492431$  y  $b_{OLS} = 0,47670009$ . Las diferencias absolutas son 0.188 para la pendiente y 0.278 para el intercepto, **órdenes de magnitud mayores** que en los casos anteriores. La brecha de optimización es astronómica: **465 563.27 %**, lo que confirma que el GD ni siquiera está cerca del fondo del valle. En cuanto a la verdad: OLS quedó a 0.33 % de la pendiente verdadera y 4.66 % del intercepto verdadero; el GD, a 12.18 % y 60.21 % respectivamente. Aquí, OLS gana en ambos coeficientes. El tiempo de GD fue  $135.6 \mu s$  y el de OLS  $132.9 \mu s$  (OLS apenas 1.99 % más rápido) porque cincuenta iteraciones manuales son baratas y el OLS tiene un costo fijo no despreciable.



**Figura 29:** Caso 8. Mil puntos, sin estandarizar,  $\alpha = 0,001$  y solo 50 iteraciones. La brecha de optimización supera 465 000 %.

```

comparativa (w*, b*, vs verdad cuando aplica, tiempos)

comparativa sobre 1000 puntos -- alfa = 0.001, iters = 50, estandarizar = NO

metodo a mano (gradiente descendiente):
w_GD = 1.31727669
b_GD = 0.19893239
ecuacion : y = 1.3173x + 0.1989
tiempo : 135.6 us (50 iteraciones)

metodo nativo (OLS via GLM.jl, referencia exacta):
w_OLS = 1.50492431
b_OLS = 0.47670009
ecuacion : y = 1.5049x + 0.4767
tiempo : 132.9 us (1 paso, ecuaciones normales)

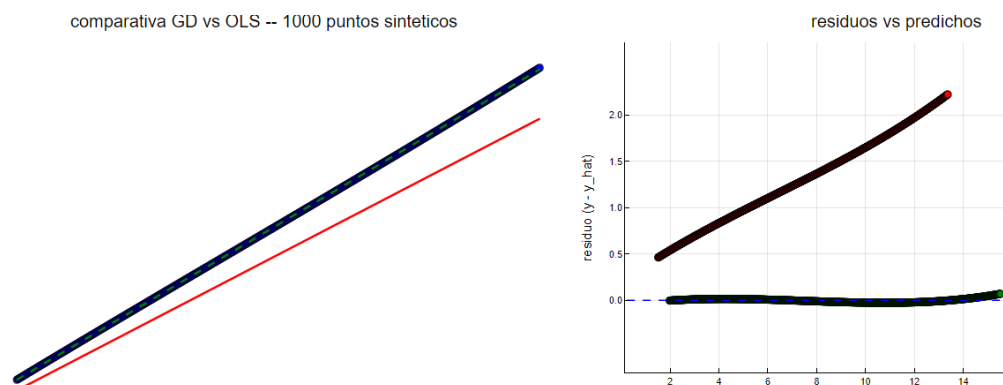
diferencias absolutas |GD - OLS|:
|w_GD - w_OLS| = 0.1876476132
|b_GD - b_OLS| = 0.277767702

comparativa porcentual (menor es mejor):
tiempo : OLS es 1.99% mas rapido
precision : OLS es 99.98% mejor
brecha : GD esta 465563.269993% por encima del minimo OLS (gap de optimizacion respecto al RSS optimo)

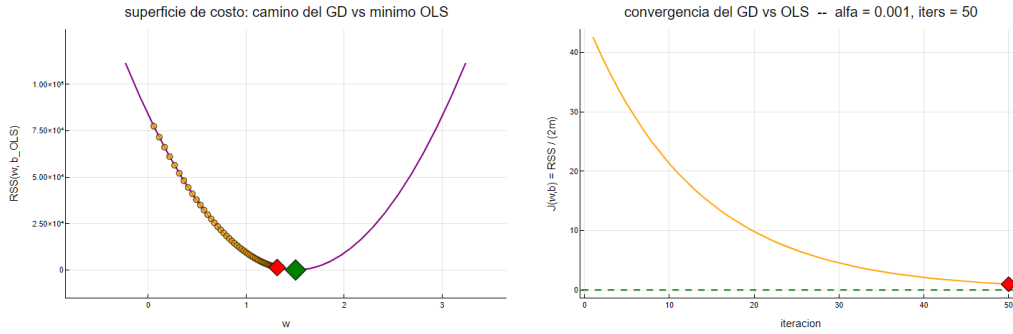
parametros VERDADEROS del dataset sintetico: a = 1.5, b = 0.5
errores absolutos respecto a la verdad:
GD : |w_GD - 1.5| = 0.1827233051 |b_GD - 0.5| = 0.3010676127
OLS : |w_OLS - 1.5| = 0.0049243081 |b_OLS - 0.5| = 0.0232999107
errores porcentuales relativos a la verdad:
GD : w = 12.181554% b = 60.213523%
OLS : w = 0.328287% b = 4.659982%
ganador en w: OLS -- ganador en b: OLS

```

**Figura 30:** Caso 8. Las dos rectas se separan visiblemente. La recta naranja del GD está claramente por debajo de la nube, mientras que la verde del OLS sigue la tendencia correcta.



**Figura 31:** Caso 8. Residuos contra valores predichos. Los residuos del GD están sistemáticamente sesgados (no centrados en cero), reflejando que la recta del GD no está ajustada al óptimo; los del OLS conservan la estructura sinusoidal del ruido determinista, sin sesgo neto.



**Figura 32:** Caso 8. Superficie de costo y curva de convergencia. La trayectoria del GD se interrumpe muy lejos del mínimo de OLS; en el panel derecho la curva del costo  $J$  todavía está descendiendo cuando se cortan las iteraciones, sin acercarse a la línea verde discontinua del óptimo.

### 3.9. Síntesis cuantitativa

La Tabla 2 reúne los números más relevantes de los ocho casos para facilitar la comparación visual.

**Tabla 2:** Síntesis cuantitativa de los ocho casos analizados. La brecha de optimización del Caso 5 figura como  $-0,00\%$  por efecto del redondeo en porcentajes cuando ambos métodos alcanzan prácticamente el mismo RSS. Los valores de los Casos 3, 6 y 7 son representativos de la corrida sobre el equipo de prueba y pueden variar ligeramente entre ejecuciones por efecto del JIT y la carga del sistema.

| Caso | $ w_{GD} - w_{OLS} $ | $ b_{GD} - b_{OLS} $ | Brecha (%)     | $t_{GD}$          | $t_{OLS}$          | Tiempo (ganador)    | Err. % w (GD / OLS) | Err. % b (GD / OLS) |
|------|----------------------|----------------------|----------------|-------------------|--------------------|---------------------|---------------------|---------------------|
| 1    | 0.0                  | 0.0                  | 0.00           | 154.6 $\mu$ s     | 104.6 $\mu$ s      | OLS (32.34 %)       | —                   | —                   |
| 2    | $5 \times 10^{-10}$  | $3,2 \times 10^{-9}$ | $\approx 0$    | 3.407 ms          | 239.0 $\mu$ s      | OLS (92.99 %)       | 0.34 / 0.34         | 4.84 / 4.84         |
| 3    | $\sim 10^{-9}$       | $\sim 10^{-9}$       | $\approx 0$    | $\sim 34$ ms      | $\sim 1$ ms        | OLS ( $\sim 97\%$ ) | 0.34 / 0.34         | 4.84 / 4.84         |
| 4    | 0.0                  | 0.0                  | 0.00           | 21.2 $\mu$ s      | 199.6 $\mu$ s      | GD (89.38 %)        | —                   | —                   |
| 5    | 0.0                  | 0.0                  | $-0,00$        | 113.1 $\mu$ s     | 245.5 $\mu$ s      | GD (53.93 %)        | 0.34 / 0.34         | 4.84 / 4.84         |
| 6    | 0.0                  | 0.0                  | 0.00           | $\sim 3$ ms       | $\sim 1$ ms        | OLS ( $\sim 60\%$ ) | 0.34 / 0.34         | 4.84 / 4.84         |
| 7    | $\sim 0,70$          | $\sim 0,22$          | $\sim 10\,000$ | $\sim 20$ $\mu$ s | $\sim 130$ $\mu$ s | GD ( $\sim 85\%$ )  | 46 / 0.33           | 48 / 4.84           |
| 8    | 0.188                | 0.278                | 465 563        | 135.6 $\mu$ s     | 132.9 $\mu$ s      | OLS (1.99 %)        | 12.18 / 0.33        | 60.21 / 4.66        |

## 4. Discusión

### 4.1. Interpretación general

El resultado más importante, y el que conviene enunciar antes que ningún otro, es uno que la propia matemática garantiza y que los experimentos confirman: **OLS y GD resuelven exactamente el mismo problema de minimización**, y en condiciones favorables (GD bien configurado, datos bien escalados, suficientes iteraciones) **entregan idéntico resultado**. El Caso 5, en el que la diferencia absoluta entre los coeficientes de los dos métodos es 0.0 con ocho dígitos de precisión, no es una excepción afortunada: es el caso típico cuando se hacen las cosas bien. Los Casos 1 y 4 entregan exactamente la misma conclusión sobre datos a mano. El Caso 2 muestra que incluso sin estandarizar, dándole al GD diez mil iteraciones, los métodos coinciden hasta el séptimo decimal.

La conclusión que de esto se desprende no es que un método sea mejor que el otro, sino que **cada uno brilla en circunstancias distintas** y cada uno paga un precio cuando se lo usa

fuera de su zona de confort.

## 4.2. Cuándo es ideal usar OLS

OLS es preferible en los siguientes contextos:

- **Cuando se necesita una respuesta exacta sin tuning.** OLS no tiene hiperparámetros: no hay  $\alpha$ , no hay número de iteraciones, no hay decisión de estandarización. La fórmula es la misma siempre, el resultado es único y reproducible. Para la persona que solo necesita *la mejor recta* sobre sus datos, OLS es la opción sin sobresaltos.
- **Cuando se necesita inferencia estadística.** GLM.jl, como cualquier librería seria de modelos lineales, entrega gratis el  $R^2$  y el  $R^2$  ajustado, los errores estándar de cada coeficiente, los estadísticos  $t$  y los  $p$ -valores, los intervalos de confianza al 95 %, el error estándar de los residuos y los criterios AIC y BIC. Toda esta maquinaria de inferencia es una ventaja que el GD no provee nativamente y que requeriría implementar varias fórmulas adicionales para reproducir [27].
- **Cuando el número de variables predictoras es pequeño** y la matriz  $X^\top X$  cabe holgadamente en memoria. En estos casos, OLS es no solo más exacto sino casi siempre más rápido.
- **Cuando los datos están bien condicionados o se desea evitar el preprocesamiento.** OLS es invariante a la escala de  $x$ ; el GD obliga a tomar decisiones sobre cómo estandarizar antes de poder confiar en el resultado.

## 4.3. Cuándo es ideal usar el descenso de gradiente

El GD es preferible (y a veces es la única opción) en los siguientes contextos:

- **Cuando el modelo no tiene solución cerrada.** Esta es la razón fundamental por la que el GD se estudia en cursos de aprendizaje automático: regresión logística, redes neuronales, máquinas de soporte vectorial con núcleos no lineales, modelos profundos. Todos ellos minimizan funciones de costo que *no tienen una fórmula tipo*  $(X^\top X)^{-1}X^\top y$ . El GD es el lenguaje común de la optimización en aprendizaje automático moderno [4, 22].
- **Cuando la dimensionalidad o el tamaño del dataset hacen impracticable formar y descomponer  $X^\top X$ .** Para millones de variables o filas, las ecuaciones normales requieren memoria prohibitiva; el GD, especialmente en sus variantes estocástica y de minilotes, puede procesar una observación o un grupo pequeño por vez [15].
- **Cuando los datos llegan en flujo continuo.** El GD se actualiza con cada nueva observación; OLS requiere recalcular toda la solución cerrada con la matriz completa.
- **Cuando el objetivo es didáctico.** El GD obliga a entender la función de costo, su gradiente, la noción de tasa de aprendizaje y la convergencia. Es el contacto inicial con la maquinaria conceptual de la optimización, transferible luego a problemas mucho más complejos [3, 5].

#### 4.4. El papel de la estandarización: la diferencia más visible

La comparación Caso 2 versus Caso 5 es el experimento que mejor ilustra el rol de la estandarización. Ambos usan exactamente los mismos cien puntos sintéticos; cambian solo el  $\alpha$ , las iteraciones y la casilla de estandarización.

- *Sin estandarizar (Caso 2):* con  $\alpha = 0,01$  e **iteraciones** = 10000, el GD alcanza prácticamente al OLS, pero tomándose más de tres milisegundos.
- *Con estandarizar (Caso 5):* con  $\alpha = 0,3$  e **iteraciones** = 300, el GD alcanza al OLS en menos de un sexto de milisegundo. Treinta veces menos iteraciones, treinta veces más rápido.

La explicación geométrica ya se introdujo en la sección de Métodos. Cuando  $x$  tiene una media muy lejos de cero y un rango amplio, la matriz  $X^T X$  tiene un número de condición alto. En el ejemplo concreto del Caso 2, con  $x$  en el rango  $[1, 10]$  sin centrar, los autovalores de  $X^T X/m$  son aproximadamente 39.3 y 0.21, dando un número de condición cercano a 187. El autovalor pequeño corresponde a una dirección en la que el GD avanza muy lentamente: con  $\alpha = 0,01$ , el factor de contracción en esa dirección es  $1 - 0,01 \cdot 0,21 \approx 0,998$ , lo que significa que cada iteración solo reduce el error en esa dirección en un 0.2 %. Mientras tanto, en la dirección del autovalor grande, el GD avanza mucho más rápido. El resultado visible en el Caso 2 es que en la superficie de costo (panel inferior izquierdo de la Figura 8) el camino del GD se ve *desplazado a la derecha del óptimo*: esto sucede porque el intercepto  $b$  se actualiza muy lentamente y, mientras tanto, la pendiente  $w$  persigue una *diana móvil* cuyo valor óptimo, dada cada estimación intermedia de  $b$ , está a la derecha de 1.5.

Estandarizar  $x$  reduce este número de condición prácticamente a 1. La superficie de costo se transforma de un canal alargado a un cuenco redondo simétrico, y **una sola tasa de aprendizaje** funciona bien para ambas direcciones simultáneamente. El GD ya no zigzaguea ni persigue dianas móviles; baja derecho al fondo en pocas iteraciones. Esto coincide con lo reportado en la documentación oficial de Google Developers en español [22] y en otras fuentes hispanas sobre normalización [17, 18, 19, 20, 21]: la normalización por z-score mejora la convergencia de los algoritmos de descenso de gradiente al uniformizar la escala de las características.

Insistimos en una idea importante para no caer en mitos: **estandarizar o no estandarizar no es una receta universal**. Depende del problema. Cuando los datos ya están en una escala homogénea, estandarizar puede ser innecesario. Cuando se necesita interpretar los coeficientes en las unidades originales de las variables, estandarizar introduce un paso de desestandarización adicional. La regla práctica es: *si se va a usar un método iterativo como el GD y las escalas de las variables son muy disímiles, estandarizar*; si se va a usar OLS, la decisión de estandarizar es indiferente para el ajuste y solo afecta a la interpretabilidad de los coeficientes.

#### 4.5. Lectura de los gráficos de residuos contra predichos

Los paneles 2 de cada caso merecen una mirada conjunta. En los Casos 1 y 4 (cinco puntos a mano) los residuos se distribuyen alrededor de cero *sin un patrón claro* porque, con tan pocos puntos, no hay suficiente estructura para que el ojo distinga señales. En los Casos 2 y 5 (cien

puntos sintéticos), los residuos forman una **curva sinusoidal nítida**: empiezan ligeramente por debajo de cero, suben hasta un máximo positivo en torno a  $\hat{y} \approx 6$ , regresan a cero, bajan a un mínimo negativo y vuelven a aproximarse a cero. Esta forma no es ruido aleatorio: es la huella exacta del término ruido( $x$ ) =  $0,001(x - 2)(x - 4)(x - 8)$ , un polinomio cúbico, que la recta no puede capturar por construcción.

Lo importante aquí es que este patrón en los residuos **aparece igual en ambos métodos**: tanto OLS como GD (cuando converge) ajustan la *mejor* recta dado el modelo lineal especificado, y esa mejor recta *no puede* eliminar una curvatura cúbica subyacente. Si en un proyecto real apareciera un patrón así, el diagnóstico correcto no sería cambiar de OLS a GD ni viceversa, sino **cambiar de modelo**: agregar un término cuadrático o cúbico en  $x$ , aplicar una transformación, o pasar a una regresión no lineal. Es exactamente el uso para el que se diseñaron los gráficos de residuos: detectar si el modelo lineal es adecuado [23, 24, 25].

#### 4.6. Sobre la brecha de optimización del Caso 8

El Caso 8 reporta una brecha de optimización del 465 563 %, una cifra que a primera vista parece anómala pero que tiene una explicación simple. La brecha se calcula como  $(RSS_{GD} - RSS_{OLS})/RSS_{OLS} \cdot 100$ . El RSS del OLS sobre mil puntos sintéticos limpios es muy pequeño (decimales bajos), porque el dataset es casi perfectamente lineal. Con un denominador chico, cualquier  $RSS_{GD}$  apreciable lleva a porcentajes enormes. Esto, lejos de ser un defecto de la métrica, es exactamente lo que se quiere: la brecha relativa magnifica diferencias que en términos absolutos podrían parecer modestas pero que, comparadas con el óptimo, son enormes. Para fines de reporte, conviene acompañar la brecha relativa con la diferencia absoluta de coeficientes, que en el Caso 8 es 0.188 para  $w$  y 0.278 para  $b$ : claramente discrepantes a simple vista.

#### 4.7. Sobre los tiempos de ejecución

Tres observaciones sobre los tiempos:

1. **En datasets muy pequeños (Casos 1 y 4)**, el OLS puede ser más lento que un GD bien configurado porque GLM.jl tiene un costo fijo (construir el *DataFrame*, parsear la fórmula, llamar internamente a las descomposiciones matriciales) que con cinco puntos representa un porcentaje no despreciable del tiempo total. Esto no debe interpretarse como una ventaja general del GD, sino como una muestra de que con datasets de juguete los tiempos son tan pequeños que están dominados por sobrecostos administrativos del lenguaje.
2. **A medida que el dataset crece (Caso 2 con cien puntos y diez mil iteraciones)**, el costo del GD escala linealmente con el número de iteraciones y con el número de puntos (cada iteración recorre todos los puntos para calcular el gradiente). En cambio, el costo del OLS escala con el tamaño del dataset una sola vez. El resultado es que el GD termina siendo notablemente más lento. La literatura técnica reporta órdenes de magnitud similares a los aquí medidos: en problemas con cientos a miles de puntos, las ecuaciones normales tienden a estar en el rango de microsegundos, mientras que el GD itera durante milisegundos.
3. **La estandarización rescata al GD**: con datos estandarizados (Caso 5), el GD necesita treinta veces menos iteraciones que sin estandarizar (Caso 2) y termina ganando en tiempo.

Esto refuerza que el preprocesamiento es parte intrínseca del diseño del GD, no un agregado opcional.

#### 4.8. Lo que GLM.jl entrega y el GD no

Es importante reconocer las limitaciones del GD implementado a mano. Aunque entrega los coeficientes  $(w^*, b^*)$  y la curva de costo, no produce nativamente las siguientes salidas que GLM.jl ofrece como parte de su API estándar:

- **Coefficiente de determinación  $R^2$  y  $R^2$  ajustado:** fracción de la varianza de  $y$  explicada por el modelo. El  $R^2$  ajustado penaliza por el número de parámetros y es más honesto en comparaciones entre modelos.
- **Errores estándar de los coeficientes:** cuantifican la incertidumbre de cada estimación.
- **Estadísticos  $t$  y  $p$ -valores:** permiten contrastar la hipótesis de que un coeficiente es estadísticamente distinto de cero.
- **Intervalos de confianza al 95 %** para cada coeficiente.
- **RSE (Residual Standard Error):** desviación típica estimada del término de error [27].
- **Criterios de información (AIC, BIC):** útiles para comparar modelos con distinto número de variables.

Para producir cualquiera de estas magnitudes a partir del GD haría falta implementar, además del propio algoritmo, las fórmulas de la matriz de covarianza de los estimadores, la inferencia bajo el supuesto de errores normales y los grados de libertad residuales. Es perfectamente factible, pero es trabajo adicional. La librería GLM.jl, simplemente, los regala. En este sentido, **OLS implementado por una librería madura no es solo un método de ajuste: es un sistema completo de inferencia estadística.**

#### 4.9. Limitaciones del estudio y autocrítica

Este trabajo, por su carácter exploratorio y didáctico, tiene limitaciones que es importante reconocer:

1. **Una sola variable predictora.** Todos los experimentos son de regresión simple  $y = wx + b$ . Las ventajas relativas de OLS y GD cambian sustancialmente al pasar a regresión múltiple con muchas variables, y especialmente con problemas de alta dimensionalidad donde la matriz  $X^\top X$  se vuelve costosa o singular. Una extensión natural del trabajo sería repetir la comparación con dos, cinco y veinte predictores.
2. **Ruido determinista, no aleatorio.** Para que las corridas sean reproducibles exactamente, el ruido es polinomial y fijo. En aplicaciones reales el ruido es estocástico y, además, las propiedades estadísticas del OLS (insesgamiento, eficiencia, BLUE bajo Gauss-Markov [1, 9]) descansan sobre supuestos de aleatoriedad e independencia que no se ponen a prueba aquí. Un experimento más completo incluiría ruido gaussiano y un análisis de la varianza de los estimadores sobre múltiples corridas.

3. **Tamaños de muestra modestos.** Aunque el dataset de 10 000 puntos comienza a mostrar la ventaja del OLS en tiempo, no llega al régimen donde el GD estocástico o por minilotes muestra su verdadero potencial. Para  $n$  en el orden de millones, ni el OLS clásico ni el GD batch implementados aquí serían apropiados.
4. **Una sola configuración de hardware.** Los tiempos medidos pertenecen a una ThinkPad T14 Gen 1 con un i5 de décima generación. En equipos con más núcleos y vectorización SIMD agresiva, los tiempos relativos podrían cambiar.
5. **Comparación bivariada de hiperparámetros.** Solo se varían  $\alpha$ , iteraciones y estandarización. Una grilla más amplia (con momentum, tasas adaptativas tipo Adam o Adagrad) mostraría que el *vanilla GD* reportado aquí está lejos de ser la mejor versión disponible del descenso de gradiente. La literatura señala que optimizadores adaptativos protegen, en parte, contra el problema de la escala de las características [22].
6. **Comparación binaria, no contra otros métodos.** El trabajo enfrenta OLS contra GD pero deja fuera ridge regression, Lasso, regresión robusta tipo Huber y descenso de gradiente estocástico, que son extensiones naturales con propiedades distintas. La comparación con esos métodos en futuros trabajos enriquecería el panorama.
7. **No se realizó validación cruzada.** Todos los reportes son sobre el mismo conjunto con el que se ajustó el modelo. Una evaluación más estricta exigiría partir los datos en entrenamiento y prueba, y reportar el error en datos no vistos.

#### 4.10. Posibles mejoras al programa

Aspectos técnicos del programa que admitirían mejora:

- **Inferencia estadística accesoria al GD:** implementar el cálculo de  $R^2$ , errores estándar y  $p$ -valores también del lado del GD, para una comparación más rica. Hoy esa información solo se reporta para el OLS de manera implícita en GLM.jl.
- **Tolerancia explícita de paro** en el GD (por ejemplo,  $|J_t - J_{t-1}| < \text{tol}$ ). Permitiría medir el tiempo *real* hasta la convergencia y no el tiempo *dado iters fijas*. En el estado actual del programa el GD siempre corre las iteraciones solicitadas, lo cual infla los tiempos si convergió antes.
- **Variantes de GD:** agregar GD estocástico, minilotes y optimizadores con tasa adaptativa, para enriquecer la comparativa.
- **Datasets con ruido gaussiano**, para evaluar propiedades estadísticas y no solo numéricas.
- **Validación cruzada y métricas fuera de muestra**, para alinear el estudio con prácticas de aprendizaje automático aplicado.
- **Regresión múltiple:** extender el modelo a varias variables predictoras y reportar cómo escalan los tiempos relativos.

#### 4.11. Futuras líneas de investigación

Con base en lo anterior, dos líneas se perfilan como naturales: (1) repetir este tipo de comparativa con la API unificada de MLJ.jl que abarca, además de OLS, modelos regularizados (Ridge, Lasso, Elastic Net) y robustos (Huber, regresión cuantil); y (2) extender el banco de pruebas a problemas no lineales en los que el GD deja de ser una alternativa al OLS y se vuelve la única opción factible, ofreciendo así un puente conceptual hacia redes neuronales y modelos profundos. Ambos ejes mantienen la pregunta original (¿cuándo conviene cada enfoque?) pero amplían el espacio donde tiene sentido formularla.

#### 4.12. Conclusión

La comparativa entre el método de mínimos cuadrados ordinarios provisto por GLM.jl y un descenso de gradiente implementado a mano confirma que ambos enfoques resuelven el mismo problema y, en condiciones adecuadas, entregan idéntico resultado. Las diferencias prácticas no están en la matemática subyacente sino en sus exigencias: OLS demanda construir y resolver un sistema lineal, lo cual es excelente cuando los datos caben en memoria y son moderadamente dimensionales; GD demanda elegir hiperparámetros y, salvo que se estandarice, sufre cuando las escalas son disímiles. La estandarización aparece, una y otra vez en los resultados, como la pieza que reconcilia al GD con el óptimo del OLS sin requerir miles de iteraciones. La librería GLM.jl, además, devuelve un paquete de inferencia estadística que rebasa el ámbito de la simple estimación puntual y que un GD implementado a mano no provee sin trabajo extra. No hay, en términos generales, un método *mejor*; hay decisiones contextuales que pueden inclinarlo todo hacia uno o hacia otro. Documentar esas decisiones, y mostrar visualmente sus consecuencias, era el propósito de este estudio.

### Declaración sobre el uso de inteligencia artificial

Durante la elaboración de este trabajo se empleó el modelo de inteligencia artificial Claude Opus 4.7 (Anthropic) como herramienta de apoyo técnico, de manera acotada y supervisada. Su uso se limitó a dos tareas: verificar y corregir los formatos de impresión de la interfaz gráfica construida con Gtk, y revisar la sintaxis del código fuente en Julia. La concepción del estudio, el diseño experimental, la implementación de los métodos, la ejecución de las pruebas, así como el análisis y la redacción de los resultados, son responsabilidad de los autores. La herramienta no generó datos experimentales ni conclusiones; su intervención se circunscribió a la depuración técnica antes señalada, y utilizado únicamente en formatos para los marcados de  $\text{\LaTeX}$ , sin participar en la redacción de los textos explicativos, interpretativos o de discusión. Se reconoce que el uso de esta herramienta puede haber contribuido a mejorar la presentación técnica del trabajo, pero no a su contenido científico ni a sus conclusiones. Se declara que el estudio se realizó con integridad académica y sin depender de la inteligencia artificial para la generación de ideas, análisis o redacción.

## Referencias

- [1] Wikipedia, “Mínimos cuadrados ordinarios,” *Wikipedia, la enciclopedia libre*. [En línea]. Disponible: [https://es.wikipedia.org/wiki/M%C3%ADnimos\\_cuadrados\\_ordinarios](https://es.wikipedia.org/wiki/M%C3%ADnimos_cuadrados_ordinarios)
- [2] CDS Institute, “Mínimos Cuadrados Ordinarios (MCO),” *Data Science Python Blog*. [En línea]. Disponible: <https://datasciencepythonblog.net/minimos-cuadrados-ordinarios-mco/>
- [3] Codificando Bits, “La Regresión Lineal en el Machine Learning,” *Codificando Bits*. [En línea]. Disponible: <https://codificandobits.com/blog/regresion-lineal/>
- [4] IBM, “¿Qué es el descenso del gradiente?,” *IBM Think*. [En línea]. Disponible: <https://www.ibm.com/mx-es/think/topics/gradient-descent>
- [5] ichi.pro, “Regresión lineal usando gradiente descendente,” *ichi.pro*. [En línea]. Disponible: <https://ichi.pro/es/regresion-lineal-usando-gradiente-descendente-166742583150897>
- [6] Google Developers, “Regresión lineal: Ejercicio de descenso de gradientes,” *Machine Learning Crash Course*. [En línea]. Disponible: <https://developers.google.com/machine-learning/crash-course/linear-regression/gradient-descent-exercise?hl=es-419>
- [7] El Laberinto de Falken, “Regresión lineal y descenso de gradiente con Python,” *elaberintodefalken.com*, mar. 2019. [En línea]. Disponible: <https://www.ellaberintodefalken.com/2019/03/regresion-lineal-descenso-de-gradiente.html>
- [8] Dialéktico, “Métricas de evaluación de modelos de regresión,” *dialektico.com*. [En línea]. Disponible: <https://dialektico.com/metricas-de-evaluacion-de-modelos-de-regresion/>
- [9] Esri, “Mínimos cuadrados ordinarios (OLS),” *Documentación de ArcGIS Pro*. [En línea]. Disponible: <https://pro.arcgis.com/es/pro-app/latest/tool-reference/spatial-statistics/ordinary-least-squares.htm>
- [10] DataCamp, “Explicación de las funciones de pérdida en el machine learning,” *DataCamp tutoriales*. [En línea]. Disponible: <https://www.datacamp.com/es/tutorial/loss-function-in-machine-learning>
- [11] IBM, “¿Qué es la función de pérdida?,” *IBM Think*. [En línea]. Disponible: <https://www.ibm.com/es-es/think/topics/loss-function>
- [12] NumXL, “MSE – Error Cuadrático Medio,” *Centro de ayuda NumXL*. [En línea]. Disponible: <https://support.numxl.com/hc/es/articles/115001223423-MSE-Error-Cuadr%C3%A1tico-Medio>
- [13] DataCamp, “Tutorial de regresión lineal en R: función lm con ejemplos,” *DataCamp tutoriales*. [En línea]. Disponible: <https://www.datacamp.com/es/tutorial/linear-regression-R>



- [26] Universidad de Cantabria, “Tema 3: Regresión lineal,” *personales.unican.es*. [En línea]. Disponible: [https://personales.unican.es/rasillad/docencia/G14/TEMA\\_3/regresion\\_lineal.html](https://personales.unican.es/rasillad/docencia/G14/TEMA_3/regresion_lineal.html)
- [27] ASTM, “Calidad del modelo,” *ASTM International*. [En línea]. Disponible: <https://www.astm.org/news/esp/calidad-del-modelo-nd21>
- [28] AI Future School, “Lenguaje Julia y su rendimiento en cálculos rápidos,” *ai-futureschool.com*. [En línea]. Disponible: <https://www.ai-futureschool.com/es/informatica/lenguaje-julia-y-su-alto-rendimiento.php>
- [29] Tokio School, “Lenguaje de programación Julia: ideal para Machine Learning,” *tokioschool.com*. [En línea]. Disponible: <https://www.tokioschool.com/noticias/lenguaje-programacion-julia/>
- [30] Wikipedia, “Julia (lenguaje de programación),” *Wikipedia, la enciclopedia libre*. [En línea]. Disponible: [https://es.wikipedia.org/wiki/Julia\\_\(lenguaje\\_de\\_programaci%C3%B3n\)](https://es.wikipedia.org/wiki/Julia_(lenguaje_de_programaci%C3%B3n))

## A. Código fuente íntegro

A continuación se incluye el contenido íntegro del archivo `comparativa_regresion.jl`, base del estudio. Se reproduce textualmente para garantizar la reproducibilidad del experimento.

```

1  using Gtk
2  using Plots
3  using GLM
4  using DataFrames
5  plotlyjs()
6  const X_3 = [1.0, 2.0, 3.0]
7  const Y_3 = [2.0, 4.0, 5.5]
8
9  const X_5 = [1.0, 2.0, 3.0, 4.0, 5.0]
10 const Y_5 = [1.5, 3.0, 4.0, 5.5, 7.0]
11
12 const PENDIENTE_VERDADERO = 1.5
13 const INTERCEPTO_VERDADERO = 0.5
14
15 function ruido_determinista(x)
16     return 0.001 * (x - 2.0) * (x - 4.0) * (x - 8.0)
17 end
18
19 const X_100 = collect(range(1.0, 10.0, length = 100))
20 const Y_100 = [PENDIENTE_VERDADERO * x + INTERCEPTO_VERDADERO +
21               ruido_determinista(x) for x in X_100]
22
23 const X_1000 = collect(range(1.0, 10.0, length = 1000))
24 const Y_1000 = [PENDIENTE_VERDADERO * x + INTERCEPTO_VERDADERO +
25                 ruido_determinista(x) for x in X_1000]

```

```

25 const X_10000 = collect(range(1.0, 10.0, length = 10000))
26 const Y_10000 = [PENDIENTE_VERDADERO * x + INTERCEPTO_VERDADERO +
    ruido_determinista(x) for x in X_10000]
27
28
29 function modelo_lineal(pendiente, intercepto, x)
30     return pendiente * x + intercepto
31 end
32
33 function funcion_costo(pendiente, intercepto, x_datos, y_datos)
34     cantidad_muestras = length(x_datos)
35     suma = sum((modelo_lineal(pendiente, intercepto, x_datos[i]) - y_datos[i])^2 for i in 1:cantidad_muestras)
36     return suma / (2.0 * cantidad_muestras)
37 end
38
39 function derivada_pendiente(pendiente, intercepto, x_datos, y_datos)
40     cantidad_muestras = length(x_datos)
41     suma = sum((modelo_lineal(pendiente, intercepto, x_datos[i]) - y_datos[i]) * x_datos[i] for i in 1:cantidad_muestras)
42     return suma / cantidad_muestras
43 end
44
45 function derivada_intercepto(pendiente, intercepto, x_datos, y_datos)
46     cantidad_muestras = length(x_datos)
47     suma = sum(modelo_lineal(pendiente, intercepto, x_datos[i]) - y_datos[i] for i in 1:cantidad_muestras)
48     return suma / cantidad_muestras
49 end
50
51 function gradiente_descendiente(x_datos, y_datos, alfa, numero_iteraciones)
52     pendiente = 0.0
53     intercepto = 0.0
54     historial_costo = Float64[]
55     historial_pendiente = Float64[]
56     historial_intercepto = Float64[]
57
58     for _ in 1:numero_iteraciones
59         w = pendiente - alfa * derivada_pendiente(pendiente, intercepto,
x_datos, y_datos)
60         b = intercepto - alfa * derivada_intercepto(pendiente, intercepto,
x_datos, y_datos)
61         pendiente = w
62         intercepto = b
63         push!(historial_costo, funcion_costo(pendiente, intercepto, x_datos,
y_datos))
64         push!(historial_pendiente, pendiente)
65         push!(historial_intercepto, intercepto)
66     end
67
68     return pendiente, intercepto, historial_costo, historial_pendiente,
historial_intercepto

```

```

69 end
70
71
72 function estandarizar_x(x_datos)
73     cantidad_muestras = length(x_datos)
74     media = sum(x_datos) / cantidad_muestras
75     varianza = sum((valor - media)^2 for valor in x_datos) /
    cantidad_muestras
76     desviacion_estandar = sqrt(varianza)
77     x_estandarizado = [(valor - media) / desviacion_estandar for valor in
    x_datos]
78     return x_estandarizado, media, desviacion_estandar
79 end
80
81 function desestandarizar_coeficientes(pendiente_estandarizada,
    intercepto_estandarizado, media, desviacion_estandar)
82     pendiente_original = pendiente_estandarizada / desviacion_estandar
83     intercepto_original = intercepto_estandarizado - pendiente_estandarizada
    * media / desviacion_estandar
84     return pendiente_original, intercepto_original
85 end
86
87
88 function ajustar_minimos_cuadrados(x_datos, y_datos)
89     tabla_datos = DataFrame(x = x_datos, y = y_datos)
90     return lm(@formula(y ~ x), tabla_datos)
91 end
92
93 function calcular_rss(valor_pendiente, valor_intercepto, x_datos, y_datos)
94     return sum((valor_pendiente .* x_datos .+ valor_intercepto .- y_datos)
    .^ 2)
95 end
96
97
98 function graficar_comparativa(x_datos, y_datos,
99     pendiente_gradiente, intercepto_gradiente,
    historial_costo, historial_pendiente,
100     pendiente_minimos_cuadrados,
    intercepto_minimos_cuadrados, rss_minimos_cuadrados,
101     titulo, alfa, iteraciones)
102
103     cantidad_iteraciones = length(historial_pendiente)
104     rango_x = collect(range(minimum(x_datos), maximum(x_datos), length =
    200))
105     y_gradiente = modelo_lineal.(pendiente_gradiente, intercepto_gradiente,
    rango_x)
106     y_minimos_cuadrados = modelo_lineal.(pendiente_minimos_cuadrados,
    intercepto_minimos_cuadrados, rango_x)
107
108     y_completo = vcat(y_datos, y_gradiente, y_minimos_cuadrados)
109     margen_vertical = (maximum(y_completo) - minimum(y_completo)) * 0.20
110     margen_horizontal = (maximum(x_datos) - minimum(x_datos)) * 0.10

```

```

111     panel_uno = scatter(x_datos, y_datos,
112         label = "datos ($(length(x_datos)) puntos)",
113         xlabel = "x",
114         ylabel = "y",
115         title = titulo,
116         markersize = 4,
117         color = :blue,
118         legend = :topleft,
119         alpha = 0.5
120     )
121     plot!(panel_uno, rango_x, y_gradiente,
122         label = "GD : y = $(round(Float64(pendiente_gradiente), digits = 4))
123 x + $(round(Float64(intercepto_gradiente), digits = 4))",
124         color = :red,
125         linewidth = 2.5
126     )
127     plot!(panel_uno, rango_x, y_minimos_cuadrados,
128         label = "OLS: y = $(round(Float64(pendiente_minimos_cuadrados),
129 digits = 4))x + $(round(Float64(intercepto_minimos_cuadrados), digits =
130 4))",
131         color = :green,
132         linewidth = 2.5,
133         linestyle = :dash
134     )
135     xlims!(panel_uno, (minimum(x_datos) - margen_horizontal, maximum(x_datos
136 ) + margen_horizontal))
137     ylims!(panel_uno, (minimum(y_completo) - margen_vertical, maximum(
138 y_completo) + margen_vertical))
139
140     y_predicho_gradiente = modelo_lineal.(pendiente_gradiente,
141 intercepto_gradiente, x_datos)
142     y_predicho_minimos_cuadrados = modelo_lineal.(
143 pendiente_minimos_cuadrados, intercepto_minimos_cuadrados, x_datos)
144     residuo_gradiente = y_datos .- y_predicho_gradiente
145     residuo_minimos_cuadrados = y_datos .- y_predicho_minimos_cuadrados
146
147     panel_dos = scatter(y_predicho_gradiente, residuo_gradiente,
148         label = "GD",
149         xlabel = "predicho y_hat",
150         ylabel = "residuo (y - y_hat)",
151         title = "residuos vs predichos",
152         color = :red,
153         markersize = 4,
154         alpha = 0.75,
155         legend = :topright
156     )
157     scatter!(panel_dos, y_predicho_minimos_cuadrados,
158 residuo_minimos_cuadrados,
159         label = "OLS",
160         color = :green,
161         markersize = 4,
162         alpha = 0.75

```

```

155 )
156 hline!(panel_dos, [0.0],
157     label = "cero",
158     color = :blue,
159     linewidth = 1.5,
160     linestyle = :dash
161 )
162
163 margen = max(abs(pendiente_minimos_cuadrados) * 0.8, abs(
pendiente_minimos_cuadrados - historial_pendiente[1]) * 1.2, 1.0)
164 eje_pendiente = collect(range(pendiente_minimos_cuadrados - margen,
pendiente_minimos_cuadrados + margen, length = 400))
165 eje_costo = [calcular_rss(valor, intercepto_minimos_cuadrados, x_datos,
y_datos) for valor in eje_pendiente]
166 panel_tres = plot(eje_pendiente, eje_costo,
167     label = "",
168     xlabel = "w",
169     ylabel = "RSS(w, b_OLS)",
170     title = "superficie de costo: camino del GD vs minimo OLS",
171     color = :purple,
172     linewidth = 2,
173     legend = :topright
174 )
175 paso_camino = max(1, cantidad_iteraciones ÷ 60)
176 indices_camino = collect(1:paso_camino:cantidad_iteraciones)
177 costos_camino_gradiente = [calcular_rss(historial_pendiente[i],
intercepto_minimos_cuadrados, x_datos, y_datos) for i in indices_camino]
178 scatter!(panel_tres, historial_pendiente[indices_camino],
costos_camino_gradiente,
179     label = "camino del GD",
180     color = :orange,
181     markersize = 4,
182     alpha = 0.75
183 )
184 scatter!(panel_tres, [pendiente_minimos_cuadrados], [
rss_minimos_cuadrados],
185     label = "OLS exacto  w* = $(round(pendiente_minimos_cuadrados,
digits = 4))",
186     color = :green,
187     markersize = 11,
188     markershape = :diamond
189 )
190 scatter!(panel_tres, [pendiente_gradiente], [calcular_rss(
pendiente_gradiente, intercepto_minimos_cuadrados, x_datos, y_datos)],
191     label = "GD final  w = $(round(pendiente_gradiente, digits = 4))",
192     color = :red,
193     markersize = 9,
194     markershape = :diamond
195 )
196
197 costo_minimos_cuadrados = rss_minimos_cuadrados / (2.0 * length(x_datos)
)

```

```

198     panel_cuatro = plot(1:length(historial_costo), historial_costo,
199         label = "J del GD",
200         xlabel = "iteracion",
201         ylabel = "J(w,b) = RSS / (2m)",
202         title = "convergencia del GD vs OLS -- alfa = $alfa, iters =
$iteraciones",
203         color = :orange,
204         linewidth = 2,
205         legend = :topright
206     )
207     hline!(panel_cuatro, [costo_minimos_cuadrados],
208         label = "J* (referencia OLS) = $(round(costo_minimos_cuadrados,
digits = 6))",
209         color = :green,
210         linewidth = 2,
211         linestyle = :dash
212     )
213     scatter!(panel_cuatro, [length(historial_costo)], [historial_costo[end
214     ]],
215         label = "J final GD = $(round(historial_costo[end], digits = 6))",
216         color = :red,
217         markersize = 8,
218         markershape = :diamond
219     )
220     figura = plot(panel_uno, panel_dos, panel_tres, panel_cuatro,
221         layout = (2, 2),
222         size = (1500, 950),
223         left_margin = 8Plots.mm,
224         right_margin = 8Plots.mm,
225         top_margin = 5Plots.mm,
226         bottom_margin = 5Plots.mm,
227         link = :none
228     )
229     display(figura)
230 end
231
232
233 function lanzar_aplicacion()
234
235     let x_calentamiento = [1.0, 2.0, 3.0], y_calentamiento = [1.0, 2.0, 3.0]
236     gradiente_descendiente(x_calentamiento, y_calentamiento, 0.01, 10)
237     ajustar_minimos_cuadrados(x_calentamiento, y_calentamiento)
238     estandarizar_x(x_calentamiento)
239 end
240
241     ventana = GtkWindow("comparativa GD vs OLS -- gradiente descendiente
vs GLM.jl", 820, 880)
242     caja = GtkBox(:v)
243     set_gtk_property!(caja, :spacing, 8)
244     set_gtk_property!(caja, :margin_top, 12)
245     set_gtk_property!(caja, :margin_bottom, 12)

```

```

246 set_gtk_property!(caja, :margin_start, 12)
247 set_gtk_property!(caja, :margin_end, 12)
248
249 etiqueta_titulo = GtkLabel(
250     "<b>comparativa GD (a mano) vs OLS (GLM.jl)</b>\n" *
251     "modelo :  $y = wx + b$  -- misma data, dos metodos de ajuste\n" *
252     "GD : iterativo, depende de alfa e iteraciones -- OLS : cerrado,
un solo paso\n" *
253     "datasets sinteticos generados con  $a = \$PENDIENTE\_VERDADERO$ ,  $b =$ 
 $\$INTERCEPTO\_VERDADERO$  (puntaje a igualar)"
254 )
255 set_gtk_property!(etiqueta_titulo, :use_markup, true)
256 set_gtk_property!(etiqueta_titulo, :justify, 2)
257
258 function fila_campo(texto_etiqueta, valor_predeterminado,
ancho_caracteres = 12)
259     fila = GtkBox(:h)
260     set_gtk_property!(fila, :spacing, 6)
261     etiqueta = GtkLabel(texto_etiqueta)
262     set_gtk_property!(etiqueta, :xalign, 0.0f0)
263     entrada = GtkEntry()
264     set_gtk_property!(entrada, :text, valor_predeterminado)
265     set_gtk_property!(entrada, :width_chars, ancho_caracteres)
266     push!(fila, etiqueta)
267     push!(fila, entrada)
268     return fila, entrada
269 end
270
271 marco_parametros = GtkFrame(" hiperparametros del GD (OLS no los usa,
su ajuste es siempre el mismo) ")
272 caja_parametros = GtkBox(:v)
273 set_gtk_property!(caja_parametros, :spacing, 6)
274 set_gtk_property!(caja_parametros, :margin_top, 6)
275 set_gtk_property!(caja_parametros, :margin_bottom, 6)
276 set_gtk_property!(caja_parametros, :margin_start, 8)
277 set_gtk_property!(caja_parametros, :margin_end, 8)
278
279 fila_alfa, entrada_alfa = fila_campo("learning rate (alfa): ", "0.01")
280 fila_iteraciones, entrada_iteraciones = fila_campo("numero de
iteraciones: ", "1000")
281
282 casilla_estandarizar = GtkCheckButton("estandarizar x para el GD (z-
score:  $\mu=0$ ,  $\sigma=1$ ) -- OLS es invariante a la escala, no lo necesita")
283 set_gtk_property!(casilla_estandarizar, :active, true)
284
285 etiqueta_nota = GtkLabel(" alfa: 0.001 a 0.05 -- iters bajos (~1000)
muestran diferencias visibles GD vs OLS; iters altos (>2000) hacen
converger ambos al mismo valor")
286 set_gtk_property!(etiqueta_nota, :xalign, 0.0f0)
287
288 for widget in [fila_alfa, fila_iteraciones, casilla_estandarizar,
etiqueta_nota]

```

```

289     push!(caja_parametros, widget)
290 end
291 push!(marco_parametros, caja_parametros)
292
293 marco_predefinidos = GtkFrame(" conjuntos de datos predefinidos (los 2
    primeros son a mano; los 3 ultimos son sinteticos con verdad conocida)
    ")
294 caja_predefinidos = GtkBox(:v)
295 set_gtk_property!(caja_predefinidos, :spacing, 6)
296 set_gtk_property!(caja_predefinidos, :margin_top, 6)
297 set_gtk_property!(caja_predefinidos, :margin_bottom, 6)
298 set_gtk_property!(caja_predefinidos, :margin_start, 8)
299 set_gtk_property!(caja_predefinidos, :margin_end, 8)
300
301 boton_3 = GtkButton("3 puntos a mano: (1, 2.0), (2, 4.0), (3, 5.5) --
    sin verdad definida")
302 boton_5 = GtkButton("5 puntos a mano: (1, 1.5), (2, 3.0), (3, 4.0), (4,
    5.5), (5, 7.0) -- sin verdad definida")
303 boton_100 = GtkButton("100 puntos sintetico:  $y = \text{\$}(\text{PENDIENTE\_VERDADERO})x$ 
    +  $\text{\$}(\text{INTERCEPTO\_VERDADERO})$  + ruido suave determinista ( $|\epsilon| < 0.1$ ), sin
    outliers)")
304 boton_1000 = GtkButton("1000 puntos sintetico: misma formula, mas denso
    -- aqui empieza a notarse el escalado")
305 boton_10000 = GtkButton("10000 puntos sintetico: misma formula, denso --
    aqui se ve la ventaja real de OLS en tiempo")
306
307 for boton in [boton_3, boton_5, boton_100, boton_1000, boton_10000]
308     push!(caja_predefinidos, boton)
309 end
310 push!(marco_predefinidos, caja_predefinidos)
311
312 marco_resultados = GtkFrame(" comparativa (w*, b*, vs verdad cuando
    aplica, tiempos) ")
313 etiqueta_resultados = GtkLabel("presiona un boton para ejecutar ambos
    metodos sobre la misma data...")
314 set_gtk_property!(etiqueta_resultados, :margin_top, 10)
315 set_gtk_property!(etiqueta_resultados, :margin_bottom, 10)
316 set_gtk_property!(etiqueta_resultados, :margin_start, 10)
317 set_gtk_property!(etiqueta_resultados, :margin_end, 10)
318 set_gtk_property!(etiqueta_resultados, :xalign, 0.0f0)
319 push!(marco_resultados, etiqueta_resultados)
320
321 for widget in [etiqueta_titulo, marco_parametros, marco_predefinidos,
    marco_resultados]
322     push!(caja, widget)
323 end
324 push!(ventana, caja)
325
326 function leer_parametros()
327     alfa = parse(Float64, strip(get_gtk_property(entrada_alfa, :text,
    String)))
328     iteraciones = parse(Int, strip(get_gtk_property(entrada_iteraciones,

```

```

: text, String)))
329     estandarizar_activado = get_gtk_property(casilla_estandarizar, :
active, Bool)
330     return alfa, iteraciones, estandarizar_activado
331 end
332
333 function formatear_tiempo(tiempo_segundos)
334     if tiempo_segundos < 1.0e-3
335         return "$(round(tiempo_segundos * 1.0e6, digits = 2)) us"
336     elseif tiempo_segundos < 1.0
337         return "$(round(tiempo_segundos * 1.0e3, digits = 3)) ms"
338     else
339         return "$(round(tiempo_segundos, digits = 4)) s"
340     end
341 end
342
343 function mostrar_comparativa(cantidad_puntos, pendiente_gradiente,
intercepto_gradiente, pendiente_minimos_cuadrados,
intercepto_minimos_cuadrados,
344                             alfa, iteraciones, estandarizar_activado,
tiempo_gradiente, tiempo_minimos_cuadrados,
345                             pendiente_verdadera, intercepto_verdadero,
rss_gradiente, rss_minimos_cuadrados)
346     function porcentaje_mejora(valor_mejor, valor_peor)
347         return valor_peor > 0 ? (1.0 - valor_mejor / valor_peor) * 100.0
: NaN
348     end
349
350     diferencia_pendiente = abs(pendiente_gradiente -
pendiente_minimos_cuadrados)
351     diferencia_intercepto = abs(intercepto_gradiente -
intercepto_minimos_cuadrados)
352     texto_estandarizacion = estandarizar_activado ? "SI (z-score)" : "NO
"
353
354     ganador_tiempo = "empate"
355     porcentaje_tiempo = 0.0
356     if abs(tiempo_gradiente - tiempo_minimos_cuadrados) > 1.0e-9
357         ganador_tiempo = tiempo_gradiente < tiempo_minimos_cuadrados ? "
GD" : "OLS"
358         tiempo_menor = min(tiempo_gradiente, tiempo_minimos_cuadrados)
359         tiempo_mayor = max(tiempo_gradiente, tiempo_minimos_cuadrados)
360         porcentaje_tiempo = porcentaje_mejora(tiempo_menor, tiempo_mayor
)
361     end
362     texto_tiempo = isfinite(porcentaje_tiempo) ? "$(ganador_tiempo) es $
(round(porcentaje_tiempo, digits = 2))% mas rapido" : "n/a"
363     ganador_precision = "empate"
364     porcentaje_precision = 0.0
365     if abs(rss_gradiente - rss_minimos_cuadrados) > 1.0e-12
366         ganador_precision = rss_gradiente < rss_minimos_cuadrados ? "GD"
: "OLS"

```

```

367         rss_menor = min(rss_gradiente, rss_minimos_cuadrados)
368         rss_mayor = max(rss_gradiente, rss_minimos_cuadrados)
369         porcentaje_precision = porcentaje_mejora(rss_menor, rss_mayor)
370     end
371     texto_precision = isfinite(porcentaje_precision) ? "$(
ganador_precision) es $(round(porcentaje_precision, digits = 2))% mejor"
: "n/a"
372
373     brecha_optimizacion = rss_minimos_cuadrados > 0 ? (rss_gradiente -
rss_minimos_cuadrados) / rss_minimos_cuadrados * 100.0 : NaN
374     texto_brecha = isfinite(brecha_optimizacion) ? "$(round(
brecha_optimizacion, digits = 6))% por encima del minimo OLS" : "n/a"
375
376     seccion_verdad = ""
377     if !isnan(pendiente_verdadera)
378         error_pendiente_gradiente = abs(pendiente_gradiente -
pendiente_verdadera)
379         error_pendiente_minimos_cuadrados = abs(
pendiente_minimos_cuadrados - pendiente_verdadera)
380         error_intercepto_gradiente = abs(intercepto_gradiente -
intercepto_verdadero)
381         error_intercepto_minimos_cuadrados = abs(
intercepto_minimos_cuadrados - intercepto_verdadero)
382         ganador_pendiente = error_pendiente_gradiente <
error_pendiente_minimos_cuadrados ? "GD" : (error_pendiente_gradiente >
error_pendiente_minimos_cuadrados ? "OLS" : "empate")
383         ganador_intercepto = error_intercepto_gradiente <
error_intercepto_minimos_cuadrados ? "GD" : (error_intercepto_gradiente >
error_intercepto_minimos_cuadrados ? "OLS" : "empate")
384
385         error_porcentual_pendiente_gradiente = pendiente_verdadera != 0
? error_pendiente_gradiente / abs(pendiente_verdadera) * 100.0 : NaN
386         error_porcentual_pendiente_minimos_cuadrados =
pendiente_verdadera != 0 ? error_pendiente_minimos_cuadrados / abs(
pendiente_verdadera) * 100.0 : NaN
387         error_porcentual_intercepto_gradiente = intercepto_verdadero !=
0 ? error_intercepto_gradiente / abs(intercepto_verdadero) * 100.0 : NaN
388         error_porcentual_intercepto_minimos_cuadrados =
intercepto_verdadero != 0 ? error_intercepto_minimos_cuadrados / abs(
intercepto_verdadero) * 100.0 : NaN
389
390         seccion_verdad = (
391             "\n" *
392             "parametros VERDADEROS del dataset sintetico: a =
$pendiente_verdadera, b = $intercepto_verdadero\n" *
393             "errores absolutos respecto a la verdad:\n" *
394             "    GD : |w_GD - $pendiente_verdadera| = $(round(
error_pendiente_gradiente, digits = 10))    |b_GD -
$intercepto_verdadero| = $(round(error_intercepto_gradiente, digits = 10))\n" *
395             "    OLS: |w_OLS - $pendiente_verdadera| = $(round(
error_pendiente_minimos_cuadrados, digits = 10))    |b_OLS -

```

```

$intercepto_verdadero| = $(round(error_intercepto_minimos_cuadrados,
396     digits = 10))\n" *
397     "errores porcentuales relativos a la verdad:\n" *
398     "    GD : w = $(round(error_porcentual_pendiente_gradiente,
    digits = 6))%    b = $(round(error_porcentual_intercepto_gradiente,
    digits = 6))%\n" *
399     "    OLS: w = $(round(
    error_porcentual_pendiente_minimos_cuadrados, digits = 6))%    b = $(
    round(error_porcentual_intercepto_minimos_cuadrados, digits = 6))%\n" *
    "    ganador en w: $ganador_pendiente    --    ganador en b:
    $ganador_intercepto\n"
400 )
401 end
402 texto_resultado = (
403     "comparativa sobre $cantidad_puntos puntos -- alfa = $alfa,
    iters = $iteraciones, estandarizar = $texto_estandarizacion\n" *
404     "\n" *
405     "metodo a mano (gradiente descendiente):\n" *
406     "    w_GD = $(round(pendiente_gradiente, digits = 8))\n" *
407     "    b_GD = $(round(intercepto_gradiente, digits = 8))\n" *
408     "    ecuacion : y = $(round(pendiente_gradiente, digits = 4))x + $
    (round(intercepto_gradiente, digits = 4))\n" *
409     "    tiempo : $(formatear_tiempo(tiempo_gradiente))    (
    $iteraciones iteraciones)\n" *
410     "\n" *
411     "metodo nativo (OLS via GLM.jl, referencia exacta):\n" *
412     "    w_OLS = $(round(pendiente_minimos_cuadrados, digits = 8))\n"
    *
413     "    b_OLS = $(round(intercepto_minimos_cuadrados, digits = 8))\n"
    *
414     "    ecuacion : y = $(round(pendiente_minimos_cuadrados, digits =
    4))x + $(round(intercepto_minimos_cuadrados, digits = 4))\n" *
415     "    tiempo : $(formatear_tiempo(tiempo_minimos_cuadrados))    (1
    paso, ecuaciones normales)\n" *
416     "\n" *
417     "diferencias absolutas |GD - OLS|:\n" *
418     "    |w_GD - w_OLS| = $(round(diferencia_pendiente, digits = 10))\n"
    *
419     "    |b_GD - b_OLS| = $(round(diferencia_intercepto, digits = 10))
    \n" *
420     "\n" *
421     "comparativa porcentual (menor es mejor):\n" *
422     "    tiempo : $texto_tiempo\n" *
423     "    precision : $texto_precision\n" *
424     "    brecha : GD esta $texto_brecha (gap de optimizacion
    respecto al RSS optimo)\n" *
425     seccion_verdad
426 )
427 set_gtk_property!(etiqueta_resultados, :label, texto_resultado)
428 end
429
430 function comparar_y_graficar(x_datos, y_datos, pendiente_verdadera,

```

```

intercepto_verdadero, titulo_grafica)
431     try
432         alfa, iteraciones, estandarizar_activado = leer_parametros()
433
434         if estandarizar_activado
435             x_para_gradiente, media_x, desviacion_x = estandarizar_x(
x_datos)
436         else
437             x_para_gradiente = x_datos
438             media_x = 0.0
439             desviacion_x = 1.0
440         end
441
442         tiempo_inicio_gradiente = time_ns()
443         pendiente_gradiente_cruda, intercepto_gradiente_crudo,
historial_costo, historial_pendiente_crudo, _ =
444             gradiente_descendiente(x_para_gradiente, y_datos, alfa,
iteraciones)
445         tiempo_gradiente = (time_ns() - tiempo_inicio_gradiente) / 1.0e9
446
447         if estandarizar_activado
448             pendiente_gradiente, intercepto_gradiente =
desestandarizar_coeficientes(pendiente_gradiente_cruda,
intercepto_gradiente_crudo, media_x, desviacion_x)
449             historial_pendiente = [valor / desviacion_x for valor in
historial_pendiente_crudo]
450         else
451             pendiente_gradiente = pendiente_gradiente_cruda
452             intercepto_gradiente = intercepto_gradiente_crudo
453             historial_pendiente = historial_pendiente_crudo
454         end
455
456         tiempo_inicio_minimos_cuadrados = time_ns()
457         modelo_minimos_cuadrados = ajustar_minimos_cuadrados(x_datos,
y_datos)
458         tiempo_minimos_cuadrados = (time_ns() -
tiempo_inicio_minimos_cuadrados) / 1.0e9
459         coeficientes = coef(modelo_minimos_cuadrados)
460         intercepto_minimos_cuadrados = coeficientes[1]
461         pendiente_minimos_cuadrados = coeficientes[2]
462         rss_minimos_cuadrados = deviance(modelo_minimos_cuadrados)
463         rss_gradiente = calcular_rss(pendiente_gradiente,
intercepto_gradiente, x_datos, y_datos)
464
465         mostrar_comparativa(length(x_datos), pendiente_gradiente,
intercepto_gradiente, pendiente_minimos_cuadrados,
intercepto_minimos_cuadrados,
466                             alfa, iteraciones, estandarizar_activado,
tiempo_gradiente, tiempo_minimos_cuadrados,
467                             pendiente_verdadera, intercepto_verdadero,
rss_gradiente, rss_minimos_cuadrados)
468         graficar_comparativa(x_datos, y_datos,

```

```

469             pendiente_gradiente, intercepto_gradiente,
historial_costo, historial_pendiente,
470             pendiente_minimos_cuadrados,
intercepto_minimos_cuadrados, rss_minimos_cuadrados,
471             titulo_grafica, alfa, iteraciones)
472         catch error_capturado
473             set_gtk_property!(etiqueta_resultados, :label, "error: $(string(
error_capturado))")
474         end
475     end
476
477     signal_connect(boton_3, "clicked") do _
478         comparar_y_graficar(X_3, Y_3, NaN, NaN, "comparativa GD vs OLS -- 3
puntos a mano")
479     end
480
481     signal_connect(boton_5, "clicked") do _
482         comparar_y_graficar(X_5, Y_5, NaN, NaN, "comparativa GD vs OLS -- 5
puntos a mano")
483     end
484
485     signal_connect(boton_100, "clicked") do _
486         comparar_y_graficar(X_100, Y_100, PENDIENTE_VERDADERO,
INTERCEPTO_VERDADERO, "comparativa GD vs OLS -- 100 puntos sinteticos")
487     end
488
489     signal_connect(boton_1000, "clicked") do _
490         comparar_y_graficar(X_1000, Y_1000, PENDIENTE_VERDADERO,
INTERCEPTO_VERDADERO, "comparativa GD vs OLS -- 1000 puntos sinteticos")
491     end
492
493     signal_connect(boton_10000, "clicked") do _
494         comparar_y_graficar(X_10000, Y_10000, PENDIENTE_VERDADERO,
INTERCEPTO_VERDADERO, "comparativa GD vs OLS -- 10000 puntos sinteticos")
495     end
496
497     showall(ventana)
498
499     if !isinteractive()
500         condicion_cierre = Condition()
501         signal_connect(ventana, "destroy") do _
502             notify(condicion_cierre)
503         end
504         @async Gtk.gtk_main()
505         wait(condicion_cierre)
506     end
507 end
508
509
510 lanzar_aplicacion()

```

**Listing 4:** Código fuente íntegro: comparativa\_regresion.jl